

Ill-Typed Programs Don't Evaluate

STEVEN RAMSAY, University of Bristol, UK

CHARLIE WALPOLE, University of Bristol, UK

We introduce two-sided type systems, which are sequent calculi for typing formulas. Two-sided type systems allow for hypothetical reasoning over the typing of compound program expressions, and the refutation of typing formulas. By incorporating a type of all values, these type systems support more refined notions of well-typing and ill-typing, guaranteeing both that well-typed programs don't go wrong and that ill-typed programs don't evaluate - that is, reach a value. This makes two-sided type systems suitable for incorrectness reasoning in higher-order program verification, which we illustrate through an application to precise data-flow typing in a language with constructors and pattern matching. Finally, we investigate the internalisation of the meta-level negation in the system as a complement operator on types. This motivates an alternative semantics for the typing judgement, which guarantees that ill-typed programs don't evaluate, but in which well-typed programs may yet go wrong.

CCS Concepts: • **Theory of computation** → *Logic and verification*; **Type theory**; **Program reasoning**.

Additional Key Words and Phrases: type systems, higher-order program verification, incorrectness

ACM Reference Format:

Steven Ramsay and Charlie Walpole. 2024. Ill-Typed Programs Don't Evaluate. *Proc. ACM Program. Lang.* 8, POPL, Article 67 (January 2024), 31 pages. <https://doi.org/10.1145/3632909>

1 INTRODUCTION

It is natural to think of a type system as a kind of proof system, whose purpose is reasoning about the behaviour of program expressions. In this view, the atomic formulas of the system are typings $M : A$, where M is a term and A a type and, in building a proof of a judgement $\Gamma \vdash M : A$, we aim to conclude an atomic formula $M : A$ under some assumptions Γ on the types of its free variables. Therefore, built into traditional type systems, but absent from most general proof systems, is a fundamental asymmetry: although we can conclude arbitrary typing formulas $M : A$, we may only make assumptions on variable typing formulas $x : B$. In this paper, we argue that type theory is enriched by removing this restriction, and there are interesting practical applications. Our *two-sided type systems* allow for making assumptions on the typing of arbitrary program expressions. In fact, they are particular kinds of sequent calculi over formulas of shape $M : A$.

The ability to reason hypothetically about the behaviour of compound terms gives us a new way to express how programs depend on their inputs. For example, we can prove that, if $x + 1$ evaluates to a natural number and $(\lambda fz. f(fz))(\lambda x. x)y$ evaluates to a natural number, then (x, y) is necessarily a pair of natural numbers:

$$x + 1 : \text{Nat}, (\lambda fz. f(fz))(\lambda x. x)y : \text{Nat} \vdash (x, y) : \text{Nat} \times \text{Nat}$$

We also remove the restriction that one must conclude exactly one typing formula on the right of the turnstile, with multiple formulas understood disjunctively (as in most sequent calculi). In

Authors' addresses: Steven Ramsay, University of Bristol, UK, steven.ramsay@bristol.ac.uk; Charlie Walpole, University of Bristol, UK, op18921@bristol.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2024 Copyright held by the owner/author(s).

ACM 2475-1421/2024/1-ART67

<https://doi.org/10.1145/3632909>

particular, one is allowed to derive an empty conclusion $\Gamma \vdash$, meaning that the assumed typing formulas Γ are inconsistent (the unit of disjunction is absurdity). For example, one can prove:

$$(\lambda fz. f(fz)) (\lambda x. x) (y, y) : \text{Nat} \vdash$$

stating that, no matter the value of y , the subject on the left *doesn't* evaluate to a natural number.

Despite this, the term above *does* evaluate, i.e. reduce to a value. By introducing a type Ok to characterise all values, we can use hypothetical judgements like the above to *prove* that terms *do not evaluate* (at all), that is they either diverge or go wrong. For example, we can prove:

$$(\lambda fz. f(fz)) (\lambda x. \text{pred}(x)) (y, y) : \text{Ok} \vdash$$

stating that the program that applies the predecessor function twice in succession to a pair will fail to evaluate. We say that such a term, to which we can assign the type Ok on the left, is *ill-typed*. In this paper, ill-typed is not merely a synonym for *untypable*: an ill-typed term can be typed (as one might expect from the suffix), but the type assignment is on the left of the turnstile.

As well as the familiar rules for concluding typings on the right of the turnstile, our systems also have rules for refuting assumed typings on the left. The key to the whole enterprise is a new kind of function type, the necessity arrow $A \multimap B$, pronounced '*A only to B*'. The necessity arrow describes functions that produce a B *only if* they are given an A as input. It is, in a sense that we make precise, the contrapositive of the usual (sufficiency) arrow $A \rightarrow B$, which guarantees that a B is produced *if* an A is given as input. The rule, (AbnR), for deducing that a function $\lambda x. M$ is of type $A \multimap B$, has a premise that is symmetrical to the usual rule for $A \rightarrow B$:

$$\begin{array}{c} \text{(AppL)} \frac{\Gamma \vdash M : A \multimap B, \Delta \quad \Gamma, N : A \vdash \Delta}{\Gamma, (MN) : B \vdash \Delta} \quad \text{(AbnR)} \frac{\Gamma, M : B \vdash x : A, \Delta}{\Gamma \vdash (\lambda x. M) : A \multimap B, \Delta} \end{array}$$

On the other hand, the rule (AppL) shows us how to decompose an application on the left, using necessity. As is typical in sequent calculi, it is instructive to think of a left rule in terms of refuting its principal formula. To *refute* that an application MN evaluates to an B , it suffices to *affirm* that M is a function that produces a B only if given an A , and then to *refute* that N is an A .

The ability to refute that terms evaluate becomes very useful when we consider more sophisticated type systems, designed for verifying stronger, behavioural properties of terms. In such systems, one faces the same difficulties as in general program verification, namely that proofs are difficult to construct automatically and, as a consequence, false positives – that is, where a perfectly good program is not well-typed¹ – are common. This is problematic because, in practice, a lot of the value of program verification tools derives from their use as a means to find bugs, i.e. true positives.

One might argue that false positives are already a feature of the mainstream type systems, as e.g. used in compilers, and are thus evidently not too onerous (though proponents of untyped programming may forcefully disagree). However, a significant difference is that, by and large, programmers understand enough of those systems to be able to predict false positives, or at least quickly diagnose and program around them. By contrast, type systems for program verification are often significantly more complex and the success of automated tools is bound up with the efficacy of *ad hoc* heuristics.

With two-sided typing, we can obtain a 'true positives' theorem: *ill-typed programs don't evaluate* in addition to the usual 'true negatives' theorem: *well-typed programs don't go wrong*. Thus, like O'Hearn's Incorrectness Logic for imperative programs [O'Hearn 2019], our two-sided typing provides some foundation for the use of type systems to predict erroneous program behaviours. Indeed, one of the original motivations for this work was to understand the basis for Erlang's highly effective Success Typing [Lindahl and Sagonas 2006].

¹A simple example that is rejected by many systems is: if true then 1 else (1, 1).

Pure Calculus Rules	
$(Id) \frac{}{\Gamma, x : A \vdash x : A, \Delta}$	
$(AppL) \frac{\Gamma \vdash M : B \multimap A, \Delta \quad \Gamma, N : B \vdash \Delta}{\Gamma, MN : A \vdash \Delta}$	$(AppR) \frac{\Gamma \vdash M : B \rightarrow A, \Delta \quad \Gamma \vdash N : B, \Delta}{\Gamma \vdash MN : A, \Delta}$
$(FixsR) \frac{\Gamma, f : A \rightarrow B, x : A \vdash M : B, \Delta}{\Gamma \vdash \text{fix } f(x). M : A \rightarrow B, \Delta}$	$(FixnR) \frac{\Gamma, f : A \multimap B, M : B \vdash x : A, \Delta}{\Gamma \vdash \text{fix } f(x). M : A \multimap B, \Delta}$

Fig. 1. Typing rules for the pure calculus.

We illustrate this in a two-sided, constrained type system designed for reasoning precisely about the shape of data structures. Following Aiken et al. [1994], our system has a type constructor for every datatype constructor, so that, for example, $[]$ is both the term constructor for the empty list and the type of the empty list, and $A :: B$ is the type of all non-empty (cons ::) lists whose head element has type A and whose tail is of type B . Using the system we can prove, for example, that the head function requires a cons with head element of type A in order to return an A , and that the map function requires non-emptiness of its list argument in order to provide a non-empty output:

$$\text{head} : \forall a. (a :: \text{Ok}) \multimap a \quad \text{map} : \forall a b. (a \multimap b) \rightarrow (a :: \text{Ok}) \multimap (b :: \text{Ok})$$

More precisely, the combination of *to* and *only to* arrows in map's type says that when given an *a* only to *b* function, it is guaranteed to behave like a function that necessarily requires a cons with element type *a* in order to produce a cons with element type *b*. Since our system can refute that the empty list is non-empty, taking the head of a list obtained by mapping the empty list is *ill-typed*:

$$\text{head } (\text{map } (\lambda x. x) []) : \text{Ok} \vdash$$

This program, which typically shows as a *false negative* in mainstream type systems for functional programming (i.e. is reported as well-typed but actually goes wrong), is proven by our system to be a true positive: the term is guaranteed to either diverge or go wrong.

1.1 Contributions

We introduce the notion of *two-sided type systems*, which are sequent calculi whose only formulas are typings $M : A$. We are particularly interested in two-sided systems for call-by-value λ -calculi with fixpoints, for which we also introduce the necessity function type.

Two Two-Sided Type Systems. To show that the idea has both theoretical and practical interest, we formulate two distinct two-sided type systems. Both systems have at their core the two-sided rules of Figure 1 for call-by-value (CBV), pure λ -calculus with fixpoints, but we add constants according to the topic of interest. Note: throughout this paper, we will take fixpoint abstractions $\text{fix } f(x). M$ as basic, and define $\lambda x. M$ as an abbreviation for the former in case f does not occur free in M ².

- In Sections 2-5 our goal is to illustrate the new kind of type-theoretic reasoning that two-sided systems enable, and to develop some of the metatheory. To this end, we take the rules for typing pure terms with fixpoints and add constants amounting to a simple, PCF-like language.

²We are grateful to one of the anonymous reviewers for this suggestion.

- In Section 6, our goal is to show the practical value of two-sided systems, and type inference. To this end, we introduce a two-sided *constrained* type system, which can be understood as a two-sided variant of certain systems in the literature that enable the automatic verification of pattern-match safety³. Starting from the typing rules for pure terms with fixpoints, we add top-level definitions, algebraic data types, and pattern matching. To make type inference as simple as possible, we formulate this system as an *intuitionistic* sequent calculus, in the sense that we restrict the consequent Δ in the rules to have size at most 1.

The Meaning of the Necessity Function Type, and Semantic Soundness. We introduce a CBV semantics for the two-sided type system for PCF. This semantics highlights an important difference between the meaning of the necessity arrow and all the other type operators in the system. In the absence of the necessity arrow, all of the types of the system can be thought of as defining safety properties (in a sense we make precise). However, there are choices of A and B for which $A \multimap B$ is not a safety property (Theorem 4.5). By restricting the use of function types on the right of the necessity arrow, we obtain a fragment in which all types again describe safety properties (Theorem 4.7). For this fragment we can prove *semantic soundness*: every provable judgement is true (Theorem 4.8). As a corollary, for this fragment we obtain both that *well-typed programs don't go wrong* and also that *ill-typed programs don't evaluate*.

Complements, Negation, and the Success Semantics. A natural question is the extent to which a two-sided judgement can be simulated using a one-sided judgement with a type-level negation. To answer this, we introduce a complement operator on types, $_c$, and show that it is inadequate, in the sense that $M : A \vdash$ and $\vdash M : A^c$ are not equivalent. This motivates an alternative, *success semantics* for two-sided judgements, in which showing that a term is typable does not preclude that the term goes wrong. Under the success semantics, from our two-sided system with complements, we can derive a one-sided (traditional) type system, which actually subsumes the original (Theorem 5.5). A preservation argument gives *syntactic soundness* for the one-sided system, which implies that ill-typed programs don't evaluate, though well-typed programs may yet go wrong (Theorem 5.7).

Application to Pattern-Match Safety, Type Inference, and Syntactic Soundness. The second of our two-sided systems allows for precise and automatic reasoning about the shape of data structures through expressive type inclusion constraints. For this two-sided system, we describe a fully automatic type inference algorithm which, given a term and two environments as input, computes a type assignment that is principal in a suitable sense (Theorem 6.8). This algorithm exploits the fact that our two-sided system has similar characteristics to traditional one-sided systems that admit computable type inference. By adapting the technique of Wright and Felleisen [1994] to two-sided systems, we prove *syntactic soundness*, from which we obtain that well-typed programs don't go wrong and ill-typed programs don't evaluate (Theorem 6.12).

1.2 Outline

The paper continues by introducing the basic ideas of two-sided typing through a simple, PCF-like programming language in Sections 2 and 3, before considering the semantics of such systems in Section 4. The relationship between the two-sided judgement and complement types is dealt with in Section 5, which introduces the success semantics and derives a one-sided system with a true positives theorem. Then, in Section 6, we then turn to an application of two-sided typing, namely automatic and precise reasoning in a functional programming language with constructors and pattern matching. Finally, Section 7 concludes with a review of related work.

³That is, that no execution of a given term can trigger a pattern match non-exhaustiveness exception at runtime.

2 A LANGUAGE AND ITS TWO-SIDED TYPING

We start by introducing a PCF-like, call-by-value programming language which we shall use to illustrate some basic considerations of two-sided typing.

Definition 2.1 (Terms). Assume a denumerable set of term variables x, y, z . We consider a simple functional programming language, whose *values*, typically V, W , and whose *terms*, typically M, N, P, Q , are defined by the following grammar:

$$\begin{aligned} V, W &::= x \mid \text{succ}^n(\text{zero}) \mid (V, W) \mid \text{fix } f(x). M \\ M, N, P, Q &::= \text{zero} \mid \text{succ}(M) \mid \text{pred}(M) \mid \text{ifz } M \text{ then } N \text{ else } P \mid \\ &\quad x \mid M N \mid \text{fix } f(x). M \mid (M, N) \mid \text{let } (x, y) = M \text{ in } N \end{aligned}$$

We consider terms identified up to renaming of bound variables and adopt the usual conventions regarding their treatment. A term with no free variables is said to be *closed*. A *term substitution*, typically θ , is finite map from term variables to terms. We say that a substitution is *closed* just if the terms in its range are closed. We will often write concrete substitutions explicitly as a list of maplets as in $[M_1/x_1, \dots, M_n/x_n]$, or $[\overline{M_i}/x_i]$ when indices are clear.

Abbreviations. It will be helpful to define some abbreviations. For each $n \in \mathbb{N}$, we write $\underline{n}$ for the *numeral* $\text{succ}^n(\text{zero})$. We write $\lambda x. M$ for $\text{fix } f(x). M$ when f does not occur free in M . The *let* construct is used to obtain the two components of a pair by matching but, in examples, we will typically always use it on the argument of an abstraction. Hence, it is helpful to define $\text{fix } f(x, y). M$ as an abbreviation for $\text{fix } f(z). \text{let } (x, y) = z \text{ in } M$. We write $\underline{\text{div}}$ as an abbreviation for $(\text{fix } f(x). f x) \underline{0}$, and $\underline{\text{id}}$ for $\lambda x. x$.

The language is essentially an applied CBV λ -calculus with fixpoints. Terms M can be tested for equality with zero using the if-zero expression $\text{ifz } M \text{ then } N \text{ else } P$, and can be deconstructed as a pair using the *let* expression $\text{let } (x, y) = M \text{ in } N$. Closed values are numerals, pairs, and (fixpoint) abstractions.

Definition 2.2 (Reduction). The *evaluation contexts* are defined by the following grammar:

$$\begin{aligned} \mathcal{E}, \mathcal{F} &::= \square \mid (\text{fix } f(x). M) \mathcal{E} \mid \mathcal{E} N \mid \text{succ}(\mathcal{E}) \mid \text{pred}(\mathcal{E}) \mid \\ &\quad (\mathcal{E}, M) \mid (V, \mathcal{E}) \mid \text{let } (x, y) = \mathcal{E} \text{ in } N \mid \text{ifz } \mathcal{E} \text{ then } N \text{ else } P \end{aligned}$$

Given a context $\mathcal{E}$, we write $\mathcal{E}[M]$ for the term obtained by replacing the hole $\square$ by M . The *one-step reduction relation*, written $M \triangleright N$, is the binary relation on (possibly open) terms obtained as the closure of the following schema under evaluation contexts.

$$\begin{aligned} (\text{IfZ}) \quad & \text{ifz } \underline{0} \text{ then } N \text{ else } P \triangleright N \\ (\text{IfS}) \quad & \text{ifz } \text{succ}(\underline{n}) \text{ then } N \text{ else } P \triangleright P \\ (\text{Let}) \quad & \text{let } (x, y) = (V, W) \text{ in } M \triangleright M[V/x, W/y] \\ (\text{PredZ}) \quad & \text{pred}(\underline{0}) \triangleright \underline{0} \\ (\text{PredS}) \quad & \text{pred}(\underline{n+1}) \triangleright \underline{n} \\ (\text{Fix}\beta) \quad & (\text{fix } f(x). M) V \triangleright M[V/x, \text{fix } f(x). M/f] \end{aligned}$$

The terms on the left-hand side of the above schema are said to be *redexes*. We write $M \triangleright^* N$ for the reflexive transitive closure of the one-step relation. A term M that cannot make a step is said to be in *normal form*. A term with no normal form is said to *diverge*. We write $M \Downarrow V$ just if closed term $M \triangleright^* V$ and we say that M *evaluates*. A term is said to be *stuck* just if it is a normal form that is not a value. A term is said to *get stuck* (or *go wrong*) just if it reduces to a stuck term.

2.1 Two-Sided Type System

We introduce a simple two-sided type system for this language. Starting from the rules for pure terms of Figure 1, we add rules for assigning types to terms headed by the constants of our language, and a rule allowing for reasoning by disjointness (which is discussed further in Section 2.2). The types of the system are either the base type Nat , or constructed using product or the two arrows.

Definition 2.3 (Types). The types, typically A, B and so on, are defined by the following grammar:

$$A, B ::= \text{Nat} \mid A \times B \mid A \rightarrow B \mid A \multimap B$$

We will label the usual arrow type, $A \rightarrow B$, the *sufficiency arrow* to distinguish it from the *necessity arrow*, $A \multimap B$. We pronounce the first of these types as ‘ A to B ’ and the second as ‘ A only to B ’⁴. We assume arrows associate to the right and products bind tighter than the other operators.

The idea of $A \multimap B$, which will be made precise in Section 4, is to classify those functions which return a B *only if* an A was supplied as input (hence the pronunciation). An example is the function $\lambda x. (x, 0)$ which can be assigned the type $\text{Nat} \multimap \text{Nat} \times \text{Nat}$. If this function returns a pair of numerals, its input must have been a numeral. On the other hand, $\lambda x. (0, 1)$ cannot be assigned this type, because if it returns a pair of numerals, it is *not necessary* that a numeral was input.

We shall not consider subtyping for this system, but it is useful to define a notion of type *disjointness*. We will discuss the motivation for this further in Section 2.2 but, intuitively, it is a weak axiomatisation of the assertion that two types do not have any values in common.

Definition 2.4 (Disjointness). We say that two types A and B are *disjoint*, and write $A \parallel B$, just if either (i) one is Nat and the other is not, or (ii) one is an arrow and the other is not, or (iii) one is a product and the other is not.

Now, we come to define the two-sided type system. The idea is that the system is a simple kind of sequent calculus whose only formulas are typings $M : A$.

Definition 2.5 (Type Assignment). A *typing formula*, or just *typing*, is a pair $M : A$ of a term M and type A . The term M is said to be the *subject* of the formula. A *typing judgement* is a pair of finite sets of typings, written $\Gamma \vdash \Delta$. A judgement is said to be *provable* (or *derivable*) according to the rules in Figure 2. We make some additional requirements on the occurrences of free variables that are omitted from the rules for typesetting reasons: in every use of (AbsR), (AbnR), (LetR), (LetL1), (LetL2) and (FixR), we require that the bound variables displayed do not occur freely in Γ or Δ .

In a traditional, one-sided type system, a judgement like $x : A, y : B \vdash P : C$ is conventionally understood as ‘if x has type A and y has type B , then P has type C ’. Here, we generalise this in two ways. First, we allow arbitrary terms on the left and not only variables. However, the judgement can be read in the same way. So $M : A, N : B \vdash P : C$ should be understood as:

‘if M has type A and N has type B , then P has type C ’.

Second, we allow for multiple typings (including none) on the right of the turnstile. These are understood disjunctively, so $M : A \vdash P : C, Q : D$ should be read as:

‘if M has type A , then *either* P has type C *or* Q has type D ’.

In particular, $M : A, N : B \vdash$, which has an empty conclusion, should be understood as ‘if M has type A and N has type B then false’, i.e. either M cannot have type A or N cannot have type B .

However, we must take care to say something about the meaning of ‘ M has type A ’ within a judgement. Since our language is call-by-value, there is an asymmetry between the left- and right-hand side. On the left-hand side of the turnstile ‘ M has type A ’ means M *evaluates to an* A ,

⁴Strictly speaking, for the English to be correct, one should pronounce it as ‘ A only, to B ’, but we do not recommend it.

Structural Rules	
$(Id) \frac{}{\Gamma, x : A \vdash x : A, \Delta}$	
Right Typing Rules	
$(ZeroR) \frac{}{\Gamma \vdash zero : Nat, \Delta}$	$(cR) \frac{\Gamma \vdash M : Nat, \Delta}{\Gamma \vdash c(M) : Nat, \Delta} c \in \left\{ \begin{array}{l} succ \\ pred \end{array} \right\}$
$(FixsR) \frac{\Gamma, f : A \rightarrow B, x : A \vdash M : B, \Delta}{\Gamma \vdash fix f(x). M : A \rightarrow B, \Delta}$	$(FixnR) \frac{\Gamma, f : A \multimap B, M : B \vdash x : A, \Delta}{\Gamma \vdash fix f(x). M : A \multimap B, \Delta}$
$(AppR) \frac{\Gamma \vdash M : B \rightarrow A, \Delta \quad \Gamma \vdash N : B, \Delta}{\Gamma \vdash MN : A, \Delta}$	$(PairR) \frac{\Gamma \vdash M : A, \Delta \quad \Gamma \vdash N : B, \Delta}{\Gamma \vdash (M, N) : A \times B, \Delta}$
$(LetR) \frac{\Gamma \vdash M : B \times C, \Delta \quad \Gamma, x : B, y : C \vdash N : A, \Delta}{\Gamma \vdash let (x, y) = M in N : A, \Delta}$	$(IfzR) \frac{\Gamma \vdash M : Nat, \Delta \quad \Gamma \vdash P : A, \Delta \quad \Gamma \vdash N : A, \Delta}{\Gamma \vdash ifz M then N else P : A, \Delta}$
Left Typing Rules	
$(Dis) \frac{\Gamma \vdash M : B, \Delta}{\Gamma, M : A \vdash \Delta} A \parallel B$	$(PairL) \frac{\Gamma, M_i : A_i \vdash \Delta}{\Gamma, (M_1, M_2) : A_1 \times A_2 \vdash \Delta} i \in \{1, 2\}$
$(AppL) \frac{\Gamma \vdash M : B \multimap A, \Delta \quad \Gamma, N : B \vdash \Delta}{\Gamma, MN : A \vdash \Delta}$	$(cL) \frac{\Gamma, M : Nat \vdash \Delta}{\Gamma, c(M) : Nat \vdash \Delta} c \in \left\{ \begin{array}{l} succ \\ pred \end{array} \right\}$
$(IfzL1) \frac{\Gamma, M : Nat \vdash \Delta}{\Gamma, ifz M then N else P : A \vdash \Delta}$	$(IfzL2) \frac{\Gamma, N : A \vdash \Delta \quad \Gamma, P : A \vdash \Delta}{\Gamma, ifz M then N else P : A \vdash \Delta}$
$(LetL1) \frac{\Gamma, N : A \vdash \Delta}{\Gamma, let (x, y) = M in N : A \vdash \Delta}$	$(LetL2) \frac{\Gamma, M : B_1 \times B_2 \vdash \Delta \quad \Gamma, N : A \vdash x_i : B_i, \Delta (\forall i)}{\Gamma, let (x_1, x_2) = M in N : A \vdash \Delta}$

Fig. 2. Two-sided type assignment.

but on the right-hand side, it means *either M diverges or M evaluates to an A* . This is made precise in Section 4, but it is useful to have some intuitions already.

For example, suppose we have some term add and we know that $\text{add} : \text{Nat} \times \text{Nat} \rightarrow \text{Nat}$. Then, we should expect that the following judgement is provable:

$$\text{succ}(\text{succ}(z)) : \text{Nat}, \text{ ifz } y \text{ then } \underline{0} \text{ else } \underline{1} : \text{Nat} \vdash \text{add}(y, z) : \text{Nat}$$

Intuitively, if $\text{succ}(\text{succ}(z))$ is a numeral, it must be that z was a numeral, and if the term $\text{ifz } y \text{ then } \underline{0} \text{ else } \underline{1}$ evaluates to a numeral, it must be that y is a numeral too (recall that the conditional of the language requires the guard to be a numeral). Therefore, $\text{add}(y, z)$ should evaluate to a numeral. This is the kind of reasoning that we are trying to capture with the rules of Figure 2.

The rules have been divided into three kinds, *structural rules*, *right rules* and *left rules*. Typically, the rules are used bottom-to-top in order to construct a derivation. The right rules should be familiar from simple type systems, and are for concluding typings on the right of the turnstile; they only differ in that they allow for multiple conclusions. One can think of the purpose of left rules as for refuting typings on the left of the turnstile. However, in practice, they are more often used as a means to extract information from assumptions.

Structural Rules. The rule (Id) should be familiar. The only difference is that now we allow for multiple formulas in the conclusion, but since they are understood disjunctively, all is well.

Left Rules. The (Dis) rule expresses a key mechanism for refuting that a term M evaluates to an A , namely by affirming that M either diverges or evaluates to a value of some disjoint type B . The motivation for this rule will be further discussed in Section 2.2.

The rule (SuccL) allows for refuting that a term of shape $\text{succ}(M)$ has type Nat . The only way that $\text{succ}(M)$ may *not* evaluate to a numeral is if M does not evaluate to a numeral, so refuting $\text{succ}(M) : \text{Nat}$ reduces to refuting $M : \text{Nat}$. The rule (PredL) is similar.

The rule (AppL) says that, to refute that MN evaluates to an A , we can do the following. First show that M either diverges or requires an input of type B in order to produce an A , then refute that N evaluates to a B . Notice the combination of affirmation and refutation required in this rule.

The rule (PairL) says that to refute that (M_1, M_2) evaluates to a value of type $A_1 \times A_2$ we need only refute that *one* component of the pair evaluates accordingly.

The rules (LetL1) and (LetL2) allow for reasoning about let expressions on the left. To refute that $\text{let } (x, y) = M \text{ in } N$ evaluates to an A , one can refute that the body evaluates to an A (independently of the values of x and y); this is (LetL1). Alternatively, one can refute that M evaluates to a pair (V, W) of the appropriate kind to allow $N[V/x, W/y]$ to evaluate to an A . We use the ‘if ... then ...’ reading of the judgement, in order to explain how the rule implements this strategy. The first two premises of (LetL2) say that, if N evaluates to an A then it must be that x was a B and that y was a C . In other words, in showing these two premises, we are showing that for N to evaluate to an A as in the conclusion, (x, y) is necessarily a value of $B \times C$. Then in the third premise we refute that M evaluates to a pair of this form.

Finally, the rules (IfZL1) and (IfZL2) are used for reasoning about the if-zero expression on the left. To refute that $\text{ifz } M \text{ then } N \text{ else } P$ evaluates to an A , we can refute that M evaluates to a numeral, as in (IfZL1). Alternatively, we can show that neither branch evaluates to an A , which is (IfZL2).

Right Rules. The rule (FixnR) allows for affirming the necessity function type of a fixpoint abstraction. Otherwise, the right rules are mostly standard, excepting that they allow for the more general kind of judgement.

In sequent calculi defined over sequences or multisets of formulas, it is sometimes necessary to add structural rules such as exchange, weakening and contraction. However, since our sequents

consist of two *sets* of formulas, only weakening is relevant. We do not have explicit weakening rules, but since all the rules allow for arbitrary contexts Γ and Δ on the left and right respectively, weakening is admissible.

LEMMA 2.6 (WEAKENING). *If $\Gamma \vdash \Delta$, then $\Gamma, M : A \vdash \Delta$ and $\Gamma \vdash M : A, \Delta$.*

Abstractions. Recall that we treat $\lambda x. M$ as an abbreviation for $\text{fix } f(x). M$, when f does not occur free in M . In light of Lemma 2.6, it is easy to see that the following typing rules are admissible:

$$\text{(AbsR)} \frac{\Gamma, x : B \vdash M : A, \Delta}{\Gamma \vdash \lambda x. M : B \rightarrow A, \Delta} \quad \text{(AbnR)} \frac{\Gamma, M : A \vdash x : B, \Delta}{\Gamma \vdash \lambda x. M : B \multimap A, \Delta}$$

Each requires that x does not occur free in Γ or Δ . Recall that we use $\text{fix } f(x, y). M$ as an abbreviation for $\text{fix } f(z).$ let $(x, y) = z$ in M . In the following it will be efficient to use the derived typing rule:

$$\text{(PFixnR)} \frac{\Gamma, f : B \times C \multimap A, M : A \vdash x : B, \Delta \quad \Gamma, f : B \times C \multimap A, M : A \vdash y : C, \Delta}{\Gamma \vdash \text{fix } f(x, y). M : B \times C \multimap A, \Delta}$$

Example 2.7. We can define addition $\text{add} = \text{fix } f(y, z). \text{if } z \text{ then } z \text{ else succ } (f(\text{pred}(y), z))$. To return a numeral, both its arguments must be numerals. First observe that, in the 'else' branch of the conditional, if we know that the recursive call requires a pair of numerals in order to produce a numeral, then z must be a numeral, we use Γ to abbreviate $\{f : \text{Nat} \times \text{Nat} \multimap \text{Nat}\}$ in:

$$\begin{array}{c} \text{(Id)} \frac{}{\Gamma \vdash f : \text{Nat} \times \text{Nat} \multimap \text{Nat}, z : \text{Nat}} \quad \text{(PairL)} \frac{\text{(Id)} \frac{}{\Gamma, z : \text{Nat} \vdash z : \text{Nat}}}{\Gamma, (\text{pred}(y), z) : \text{Nat} \times \text{Nat} \vdash z : \text{Nat}} \\ \text{(AppL)} \frac{}{\Gamma, f(\text{pred}(y), z) : \text{Nat} \vdash z : \text{Nat}} \\ \text{(SuccL)} \frac{}{\Gamma, \text{succ}(f(\text{pred}(y), z)) : \text{Nat} \vdash z : \text{Nat}} \end{array}$$

The same can be said in the 'then' branch of the conditional, since the 'then' branch actually returns z . So we can use (IfZL2) to conclude that z is certainly a numeral whenever the conditional expression evaluates to a numeral:

$$\text{(IfZL2)} \frac{\text{(Id)} \frac{}{\Gamma, z : \text{Nat} \vdash z : \text{Nat}} \quad \dots \quad \Gamma, \text{succ}(f(\text{pred}(y), z)) : \text{Nat} \vdash z : \text{Nat}}{\Gamma, \text{if } z \text{ then } z \text{ else succ}(f(\text{pred}(y), z)) : \text{Nat} \vdash z : \text{Nat}}$$

Abbreviating the 'else' branch for brevity, we can obtain $y : \text{Nat}$ from the fact that the conditional branches on y , using (IfZL1), and conclude using our derived rule (PFixnR) for recursive functions requiring pairs as their input:

$$\text{(PFixnR)} \frac{\text{(IfZL1)} \frac{\text{(Id)} \frac{}{\Gamma, y : \text{Nat} \vdash y : \text{Nat}}}{\Gamma, \text{if } y \text{ then } z \text{ else } \dots : \text{Nat} \vdash y : \text{Nat}} \quad \dots \quad \Gamma, \text{if } y \text{ then } z \text{ else } \dots : \text{Nat} \vdash z : \text{Nat}}{\vdash \text{add} : \text{Nat} \times \text{Nat} \multimap \text{Nat}}$$

Example 2.8. We can define the higher-order combinator $\text{twice} = \lambda f x. f(f x)$. It has the following property. For any type A , it guarantees to map all functions of type $A \multimap A$ to an output function of the same type. We abbreviate $\Gamma = \{f : A \multimap A\}$ in the proof:

$$\begin{array}{c} \text{(Id)} \frac{}{\Gamma \vdash f : A \multimap A, x : A} \quad \text{(AppL)} \frac{\text{(Id)} \frac{}{\Gamma \vdash f : A \multimap A, x : A} \quad \text{(Id)} \frac{}{\Gamma, x : A \vdash x : A}}{\Gamma, f x : A \vdash x : A} \\ \text{(Id)} \frac{}{\Gamma, f(f x) : A \vdash x : A} \\ \text{(AbnR)} \frac{}{\Gamma \vdash \lambda x. f(f x) : A \multimap A} \\ \text{(AbsR)} \frac{}{\vdash \lambda f x. f(f x) : (A \multimap A) \rightarrow A \multimap A} \end{array}$$

$$\begin{array}{c}
\text{(ZeroL)} \frac{}{\Gamma, \text{zero} : A \vdash \Delta} A \neq \text{Nat} \qquad \text{(cL2)} \frac{}{\Gamma, c(M) : A \vdash \Delta} A \neq \text{Nat} \\
\qquad \qquad \qquad c \in \{\text{succ}, \text{pred}\} \\
\\
\text{(PairL2)} \frac{}{\Gamma, (M, N) : A \vdash \Delta} A \text{ not a product} \qquad \text{(FixL)} \frac{}{\Gamma, \text{fix } f(x). M : A \vdash \Delta} A \text{ not an arrow}
\end{array}$$

Fig. 3. Alternative disjointness rules.

Example 2.9. We can prove that applying predecessor twice to a pair does *not* evaluate to a natural number. From the foregoing example, we know that twice $(\lambda x. \text{pred}(x))$ is guaranteed to behave like a function that, to produce a numeral, requires a numeral as input:

$$\begin{array}{c}
\frac{\dots}{\vdash \text{twice} : (\text{Nat} \multimap \text{Nat}) \rightarrow \text{Nat} \multimap \text{Nat}} \quad \frac{\text{(Id)} \frac{}{x : \text{Nat} \vdash x : \text{Nat}}}{\text{(PredL)} \frac{}{\text{pred}(x) : \text{Nat} \vdash x : \text{Nat}}} \\
\text{(AppR)} \frac{\vdash \text{twice} : (\text{Nat} \multimap \text{Nat}) \rightarrow \text{Nat} \multimap \text{Nat} \quad \text{(AbnR)} \frac{}{\vdash (\lambda x. \text{pred}(x)) : \text{Nat} \multimap \text{Nat}}}{\vdash \text{twice } (\lambda x. \text{pred}(x)) : \text{Nat} \multimap \text{Nat}}
\end{array}$$

Note, since our predecessor is a constant of fixed arity, we need to wrap it in an abstraction to provide it as an input. However, a pair is not a numeral, so we can use disjointness:

$$\begin{array}{c}
\frac{\dots}{\vdash \text{twice } (\lambda x. \text{pred}(x)) : \text{Nat} \multimap \text{Nat}} \quad \frac{\text{(Dis)} \frac{}{\vdash (\underline{0}, \underline{1}) : \text{Nat} \times \text{Nat}}}{(\underline{0}, \underline{1}) : \text{Nat} \vdash} \\
\text{(AppL)} \frac{\vdash \text{twice } (\lambda x. \text{pred}(x)) : \text{Nat} \multimap \text{Nat} \quad (\underline{0}, \underline{1}) : \text{Nat} \vdash}{\text{twice } (\lambda x. \text{pred}(x)) (\underline{0}, \underline{1}) : \text{Nat} \vdash}
\end{array}$$

In fact, not only does twice $(\lambda x. \text{pred}(x)) (\underline{0}, \underline{1})$ not evaluate a numeral, it does not evaluate at all – it goes wrong. In the next section we enrich our system with the means to express this

2.2 Further Discussion of the Rules

Notice that the (ZeroR) rule and the two fixpoint rules (FixsR) and (FixnR) have no corresponding rule on the left-hand side, the left rules (predL) and (succL) only concern the case when the type is Nat, and the left rule (PairL) only concerns the case when the type is a product⁵.

To understand the situation better, let us consider each in turn. To refute $\text{zero} : A$, it must be that $A \neq \text{Nat}$, so we might expect a rule like (ZeroL) in Figure 3. Similarly, to refute a formula $c(M) : A$ (for $c \in \{\text{succ}, \text{pred}\}$), either (i) $A = \text{Nat}$ and then it must be that we can refute that M is a number, as in the existing rule (cL), or (ii) it must be that $A \neq \text{Nat}$. Hence, we appear to be missing a rule such as (cL2) in Figure 3 above. A similar analysis leads us to conclude that we are missing a rule for refuting formulas $(M, N) : A$ when A is not a product type, such as (PairL2) of Figure 3.

All the rules from Figure 3 that we have noted as missing express a form of reasoning based on disjointness of types: one can refute $\text{zero} : A$, $\text{succ}(M) : A$ and $\text{pred}(M) : A$ whenever A is a type disjoint from Nat, one can refute $(M, N) : A$ whenever A is a type disjoint from any product type. It just happens that, in this system, A being disjoint from Nat simply means that $A \neq \text{Nat}$.

However, reasoning based on disjointness of types makes sense for all terms M , and not only those that are introduction forms. In principle, it should be possible to refute any typing $M : A$ on the basis of affirming a typing $M : B$ for some type B disjoint from A . Indeed, this form of reasoning

⁵This can be considered unsatisfactory because, for example, unlike when we are affirming $\text{succ}(M) : A$ in the consequent, to refute $\text{succ}(M) : A$ (i.e. on the left), it need not be the case that $A = \text{Nat}$. Similar remarks apply for pred and pair values.

is closely related to that embodied in the success typing paradigm [Lindahl and Sagonas 2006]. In success typing, if a function M can be assigned the *success type* $A \rightarrow B$, then it means that the domain of f is wholly contained within A . Therefore, we can be sure that f fails whenever it is applied to a value whose type is disjoint from A .

Thus, we formulated our system with a single rule (Dis), expressing a general notion of reasoning based on disjointness, instead of a number of special cases for each introduction form, as in Figure 3. However, in expressive power, the two are actually incomparable. On the one hand, the rules of Figure 3 are applicable even when the subject M of the principal formula would not be well-typed. For example, $\text{succ}(\text{id}) : \text{Nat} \times \text{Nat}$ can be refuted using the disjointness rules of Figure 3, but not using (Dis). On the other hand, (Dis) allows for reasoning based on disjointness even when the subject of the principal formula is not an introduction form. For example, $(\lambda xy. x) 2 3 : \text{Nat} \times \text{Nat}$ can be refuted using (Dis), but not if we were to take the rules of Figure 3 instead of (Dis). Of course, if one is interested in the most powerful system possible, then one can throw all the rules in together.

Now, let us consider refuting a typing of shape $\text{fix } f(x). M : A$. By a similar analysis, we should expect that either (i) A is not an arrow at all; or (ii) A is a sufficiency arrow, say $B \rightarrow C$, but then there is a B -value V such that $M[V/x]$ is not a C ; or (iii) A is a necessity arrow, say $B \multimap C$, but then there is a non- B value V such that $M[V/x]$ is a C . For (i), it seems we should expect a rule like (FixL) of Figure 3, but instead we use the (Dis) rule, as discussed above. For case (ii), it seems we should have a rule:

$$\frac{\Gamma \vdash V : B, \Delta \quad \Gamma, f : B \rightarrow C, M[V/x] : C \vdash \Delta}{\Gamma, \text{fix } x. M : B \rightarrow C \vdash \Delta}$$

However, this rule would be unsound. Suppose we have some B value V such that $M[V/x]$ does not evaluate to a C , and suppose, for simplicity, that M does not contain f (so the fixpoint abstraction is just an ordinary λ -abstraction). This rule would allow us to conclude that $\lambda x. M$ is not a $B \rightarrow C$ function. However, it is not enough to know that $M[V/x]$ does not evaluate to a C , because this includes the possibility that $M[V/x]$ diverges, and this is a perfectly acceptable outcome for any $B \rightarrow C$ function. For example, using this rule we could refute $\lambda x. \text{div} : \text{Nat} \rightarrow \text{Nat}$, i.e. prove the judgement $\lambda x. \text{div} : \text{Nat} \rightarrow \text{Nat} \vdash$, but this is absurd because $\lambda x. \text{div}$ is a $\text{Nat} \rightarrow \text{Nat}$ function, according to the usual understanding of this type (and our semantics in Section 4). The situation for refuting typings of shape $\text{fix } f(x). M : B \multimap C$, i.e. case (iii) above, is analogous.

Finally, although we formulated our rule (Id) rule with principal formula $x : A$ on both sides, it would be sound to take the more general $\Gamma, M : A \vdash M : A, \Delta$ as axiom instead. This would be more powerful, e.g. $x y : A \vdash x y : A$, but it takes us further away from traditional one-sided type systems than we want to go in this paper.

3 THE TYPE OF VALUES AND ILL TYPEDNESS

One may wonder whether there is any use in being able to reason hypothetically about typing. A compelling answer can be given once we introduce a new type Ok , with meaning ‘evaluates’.

Definition 3.1. We extend the grammar of types with the constant Ok and extend the typing rules to include those given in Figure 4.

The rule (OkVarR) expresses the fact that, in call-by-value, we may assume that all variables denote values. The rules (OkL) and (OkR) express the fact that Ok is the type of all values. To conclude that $M : \text{Ok}$, it suffices to show $M : A$ since every type A represents some non-empty set of values. To refute that M evaluates to an A , it suffices to refute that it evaluates at all. Rules (OkApL1) and (OkApL2) express the fact that, for an application MN to evaluate, requires that M evaluates to a function (see Remark 3.1) and that N evaluates. Rules (OkSL) and (OkPL) express the fact that the

Typing Rules for Ok		
$(OkVarR) \frac{}{\Gamma \vdash x : Ok, \Delta}$	$(OkL) \frac{\Gamma, M : Ok \vdash \Delta}{\Gamma, M : A \vdash \Delta}$	$(OkR) \frac{\Gamma \vdash M : A, \Delta}{\Gamma \vdash M : Ok, \Delta}$
$(OkApL1) \frac{\Gamma, M : Ok \multimap A \vdash \Delta}{\Gamma, MN : Ok \vdash \Delta}$	$(OkApL2) \frac{\Gamma, N : Ok \vdash \Delta}{\Gamma, MN : Ok \vdash \Delta}$	
$(OkSL) \frac{\Gamma, M : Nat \vdash \Delta}{\Gamma, succ(M) : Ok \vdash \Delta}$	$(OkPL) \frac{\Gamma, M : Nat \vdash \Delta}{\Gamma, pred(M) : Ok \vdash \Delta}$	$(OkPrL) \frac{\Gamma, M_i : Ok \vdash \Delta}{\Gamma, (M_1, M_2) : Ok \vdash \Delta}$

Fig. 4. Type Assignment with Ok

successor and predecessor are only defined on numerals. In combination with (OkL), these rules subsume (SuccL) and (PredL) respectively. Finally, (OkPrL) allows to refute that (M_1, M_2) evaluates by refuting that one of the components evaluates.

Remark 3.1. Since Ok is the set of all values, the set of all (fixpoint) abstractions can be represented by any type of shape $Ok \multimap A$. Indeed, using (FixnR) and (OkVarR) we can prove $\vdash \text{fix } f(x). M : Ok \multimap A$.

For example, if we use (OkPL) instead of (PredL) in the first derivation of Example 2.9 we can show that $\vdash \text{twice}(\lambda x. \text{pred}(x)) : Nat \multimap Ok$. Then, using (AppL) and (Dis), we can conclude $\text{twice}(\lambda x. \text{pred}(x))(\underline{0}, \underline{1}) : Ok \vdash$.

In Example 2.7, we proved that add requires both its arguments to be numerals in order to return a numeral. However, we cannot prove that it requires both its arguments to be numerals in order to evaluate. To see why, suppose the body of the function, $\text{ifz } y \text{ then } z \text{ else succ}(f(\text{pred}(y), z))$, evaluates for some actual parameters y and z . Clearly, y is required to be a numeral since it appears in the guard. However, z is *not* required to be a numeral by the branches of the conditional. In particular, the ‘then’ branch yields $z : Ok \not\vdash z : Nat$. However, there is a good reason, it is possible for an application of the function to successfully evaluate even when the second argument is not a numeral! For example, $\text{add}(\underline{0}, \lambda x. x) \Downarrow \lambda x. x$. The best we can do is prove $\vdash \text{add} : Nat \times Ok \multimap Ok$.

Ill-Typed. Some authors use the phrase ‘ill-typed’ as synonymous with untypable, but we will use it to mean that a term is provably not ok (as is suggestive of the suffix *-typed*).

Definition 3.2 (Ill-typed and Well-typed). Suppose M is a closed term.

- We say that M is *ill-typed* just if $M : Ok \vdash$.
- We say that M is *well-typed* just if $\vdash M : Ok$.

Of course, due to the (OkR) rule, the definition of well-typed subsumes the usual one. Note that M being well-typed does not imply that M evaluates, only that M does not get stuck: having formula $M : A$ on the right of the turnstile only means that M either diverges or M evaluates to an A (see Section 4 for details). On the other hand, proving that a closed term M is ill-typed can be understood as proving that M *does not* evaluate: $M : Ok \vdash$ means that M does not evaluate to a value in Ok, so M either diverges or goes wrong. The following example is ill-typed only because it will diverge.

We use Γ to abbreviate $\{f : \text{Nat} \multimap \text{Ok}\}$:

$$\begin{array}{c}
 \text{(Id)} \frac{}{\Gamma \vdash f : \text{Nat} \multimap \text{Ok}} \quad \text{(Id)} \frac{}{\Gamma, x : \text{Nat} \vdash x : \text{Nat}} \quad \text{(Id)} \frac{}{y : \text{Nat} \vdash y : \text{Nat}} \\
 \text{(AppL)} \frac{}{\Gamma, fx : \text{Ok} \vdash x : \text{Nat}} \quad \text{(AbsR)} \frac{}{\vdash \lambda y. y : \text{Nat} \rightarrow \text{Nat}} \\
 \text{(FixnR)} \frac{}{\vdash \text{fix } f(x). f x : \text{Nat} \multimap \text{Ok}} \quad \text{(Dis)} \frac{}{\lambda y. y : \text{Nat} \vdash} \\
 \text{(AppL)} \frac{}{(\text{fix } f(x). f x) (\lambda y. y) : \text{Ok} \vdash}
 \end{array}$$

4 SEMANTICS OF TYPE ASSIGNMENT

In Sections 2 and 3, we gave some intuitions about how we want to understand the *meaning* of types and the typing judgement. We now make this precise.

Definition 4.1 (Semantics of Types). Let Vals_0 be the set of all closed values. We interpret types as certain sets of closed, well-behaved terms:

$$\begin{aligned}
 \llbracket \text{Ok} \rrbracket &= \text{Vals}_0 \\
 \llbracket \text{Nat} \rrbracket &= \{0, 1, 2, \dots\} \\
 \llbracket A \times B \rrbracket &= \{ (V, W) \mid V \in \llbracket A \rrbracket, W \in \llbracket B \rrbracket \} \\
 \llbracket A \rightarrow B \rrbracket &= \{ \text{fix } f(x). M \mid \forall V \in \text{Vals}_0. V \in \llbracket A \rrbracket \Rightarrow M[V/x][\text{fix } f(x). M/f] \in \mathcal{T}_\perp \llbracket B \rrbracket \} \\
 \llbracket A \multimap B \rrbracket &= \{ \text{fix } f(x). M \mid \forall V \in \text{Vals}_0. M[V/x][\text{fix } f(x). M/f] \in \mathcal{T} \llbracket B \rrbracket \Rightarrow V \in \llbracket A \rrbracket \} \\
 \mathcal{T} \llbracket A \rrbracket &= \{ M \mid M \Downarrow V, V \in \llbracket A \rrbracket \} \\
 \mathcal{T}_\perp \llbracket A \rrbracket &= \{ M \mid M \in \mathcal{T} \llbracket A \rrbracket \vee M \Uparrow \}
 \end{aligned}$$

The idea is that $\llbracket A \rrbracket$ is the set of all closed values of type A , $\mathcal{T} \llbracket A \rrbracket$ is the set of all terms that would evaluate to a value of type A and $\mathcal{T}_\perp \llbracket A \rrbracket$ is the set of all terms that either evaluate to a value of type A or diverge. It follows that $\mathcal{T}_\perp \llbracket \text{Ok} \rrbracket$ is the set of closed terms that do not go wrong. As usual, the meaning of the sufficiency arrow $A \rightarrow B$ is as the set of all functions that guarantee to map each A value to a B value or diverge in the process. The meaning of the necessity arrow $A \multimap B$ is as the set of all functions that require a value from A in order to eventually return a value from B .

Remark 4.1. It seems there is an asymmetry between the semantics of the two arrows, but this is only because of the bias towards values induced by CBV. The two implications are equivalent to:

$$\begin{aligned}
 V \in \mathcal{T} \llbracket A \rrbracket &\Rightarrow M[V/x][\text{fix } f(x). M/f] \in \mathcal{T}_\perp \llbracket B \rrbracket \\
 M[V/x][\text{fix } f(x). M/f] \in \mathcal{T} \llbracket B \rrbracket &\Rightarrow V \in \mathcal{T}_\perp \llbracket A \rrbracket
 \end{aligned}$$

The relationship between the two function types will become clearer in Section 5, once we introduce the success semantics. We discuss the choice of CBV semantics in Section 4.2.

A consequence of the semantics of necessity is that it can be a little subtle to express properties of interest. Given a function $\lambda x. M$, its membership in some type $\llbracket A \multimap B \rrbracket$ becomes trivial whenever there is no value V such that $M[V/x] \in \mathcal{T} \llbracket B \rrbracket$. This can occur, for example, when M simply never produces an F . Thus we have: $\lambda x. \text{pred}(x) \in \llbracket \text{Nat} \times \text{Nat} \multimap \text{Nat} \times \text{Nat} \rrbracket$, because $\lambda x. \text{pred}(x)$ does not return pairs. This phenomenon can often be exploited in the proof system, using the disjointness rule. For example, using (Dis) we can prove $\vdash \lambda x. \text{pred}(x) : \text{Nat} \times \text{Nat} \multimap \text{Nat} \times \text{Nat}$.

Now, we turn to the semantics of the typing judgement. In CBV, when analysing the body M of an abstraction $\lambda x. M$, one can assume that any actual parameter that gets bound to x must have evaluated. Hence, in the denotational semantics of CBV programming languages, it is standard to interpret a judgement $x : A \vdash M : B$ as a function from *values* of type A to *computations* of type B – where the latter may diverge [Winskel 1993]. In our two-sided system we can have terms on the left-hand side that are not themselves values, so types cannot be interpreted as sets of values, but

we do want to retain this essential asymmetry of CBV. In fact, as we discuss further in Section 4.2, the soundness of much of the system depends on it. Thus, we will define the semantics so that $M : A$ on the left means that M is a term that evaluates to an A , and $M : A$ on the right means that M is a term that either evaluates to an A or diverges.

Definition 4.2 (Semantics of Judgements). A valuation, θ , is just a closed substitution.

- Given an atomic formula $M : A$, we say that a valuation θ *satisfies the formula on the left* just if $M\theta \in \mathcal{T}[[A]]$. We say that θ *satisfies the formula on the right* just if $M\theta \in \mathcal{T}_\perp[[A]]$.
- We say that a valuation θ *satisfies a set of formulas Γ on the left*, just if θ satisfies each formula in Γ on the left. A valuation θ *satisfies a set of formulas Δ on the right*, just if there is a formula in Δ that is satisfied on the right.
- We say that a judgement $\Gamma \vdash \Delta$ is *true*, written $\Gamma \models \Delta$, just if all valuations θ that satisfy Γ on the left, satisfy Δ on the right.

To keep the development smooth, whenever we talk about the satisfaction of formulas by some valuation θ , we will assume that θ closes all the terms involved (i.e. its domain is sufficiently large).

Notice from the first of the three clauses that divergence is always allowed on the right, but never on the left. So, for example, valuation $[\underline{1}/x]$ satisfies $\text{ifz } x \text{ then } \underline{0} \text{ else } \text{div} : \text{Nat}$ on the right but not on the left. Then, notice also the asymmetry in the semantics of the judgement: $\Gamma \models \Delta$ just if all valuations that satisfy *all* formulas in Γ on the left, satisfy *some* formula from Δ on the right.

4.1 Semantic Soundness for Safety Properties

By considering the semantics of types, we can draw a sharp distinction between the necessity arrow and all the other (more usual) types of the system. To see this, we will need to look in more detail at the behaviour of fixpoints in the system, and so we introduce the following *fixpoint approximants* (see, e.g. Bradfield and Walukiewicz [2018] for the use of approximants in mu-calculus):

Definition 4.3. Given a fixpoint term $\text{fix } f(x). M$, the following family of terms, indexed by $n \in \mathbb{N}$, are its *fixpoint approximants*.

$$\begin{aligned} \text{fix}^0 f(x). M &= \lambda x. \text{div} \\ \text{fix}^{n+1} f(x). M &= \lambda x. M[\text{fix}^n f(x). M / f] \end{aligned}$$

Note: we are not introducing a new piece of syntax, but merely an abbreviation that will be useful in the sequel. Each approximant is simply a finite unfolding of the possibly infinite fixpoint computation. The idea is to think of a term $C[\text{fix } f(x). M]$ containing a fixpoint as being finitely approximated by each member of the family $C[\text{fix}^n f(x). M]$.

As is common, we think of types as defining properties, with a closed term M satisfying a property A whenever $M \in \mathcal{T}_\perp[[A]]$. In the theory of program verification, it is standard to distinguish between two kinds of properties, safety and liveness. The former is typically characterised by (i) being downwards closed in the termination order – that is, if a program satisfies a safety property, then so does any program that terminates less often, and (ii) the existence of finite counterexamples – whenever a program does not satisfy a safety property, there is a finite trace to act as witness. We make precise this idea for our type system in the following:

Definition 4.4. For closed terms N and P , write $P \lesssim N$ just if P normalises implies N normalises and then they have the same normal form. Then we say that a type A defines a *safety property* just if the following are both true:

- (S1) If $C[N] \in \mathcal{T}_\perp[[A]]$ and $P \lesssim N$, then $C[P] \in \mathcal{T}_\perp[[A]]$.
- (S2) If $C[\text{fix } f(x). M] \notin \mathcal{T}_\perp[[A]]$, then there is some k for which $C[\text{fix}^k f(x). M] \notin \mathcal{T}_\perp[[A]]$.

It is possible to show that, if we exclude the necessity arrow, then all types of the system define safety properties. However, with the necessity arrow, this is no longer the case. In fact, we can find counterexamples to both aspects of safety.

THEOREM 4.5. *There are types A and B such that $A \multimap B$ is not a safety property. In particular:*

- *The type $\text{Nat} \multimap \text{Nat} \rightarrow \text{Nat}$ does not satisfy (S1).*
- *The type $\text{Nat} \multimap \text{Nat} \rightarrow \text{Nat} \multimap \text{Nat} \rightarrow \text{Nat}$ does not satisfy (S2).*

However, by inspection of these proofs, it is evident that allowing function types on the right of the necessity arrow is a key feature. We will now define a fragment of our type system in which types are restricted so as to always define safety properties.

Definition 4.6 (Safety Fragment). The *safety fragment* consists of restricting the two-sided system to judgements only involving the following types:

$$\begin{aligned} F &::= \text{Nat} \mid \text{Ok} \mid F_1 \times F_2 \\ A, B &::= \text{Nat} \mid \text{Ok} \mid A \times B \mid A \rightarrow B \mid A \multimap B \end{aligned}$$

We call types F the *finitely verifiable* types, because we can show that whenever a term inhabits a finitely verifiable type, there is already a finite approximation of the term that inhabits the type. In the safety fragment, only finitely verifiable types are allowed on the right-hand side of necessity arrows. This is enough to recover safety:

THEOREM 4.7 (SAFETY PROPERTIES). *Every type in the safety fragment defines a safety property.*

The fact that types define safety properties is very helpful for the purpose of analysing the system. The contrapositive of (S2) gives us a principle that we can use in order to show that term satisfies a type: if $\text{fix}^n f(x). M \in \mathcal{T}_\perp[[A]]$ for all n , then it follows that $\text{fix} f(x). M \in \mathcal{T}_\perp[[A]]$. We use this principle to show soundness, i.e. that every provable judgement is true.

THEOREM 4.8 (SEMANTIC SOUNDNESS). *In the safety fragment, if $\Gamma \vdash \Delta$ then $\Gamma \models \Delta$.*

This formulation of soundness is standard for a sequent calculi [Ebbinghaus et al. 1994], but there is another, more syntactical notion of soundness that is standard in the literature for type systems, namely ‘progress and preservation’ [Wright and Felleisen 1994]. Hence, we call this result *semantic soundness* and the latter kind, which we will obtain in Sections 5 and 6, *syntactic soundness*. As corollary, we obtain:

COROLLARY 4.9. *Suppose M is a closed term. In the safety fragment:*

- *If M is well typed, then M does not go wrong.*
- *If M is ill typed, then M does not evaluate.*

At the time of writing, we do not have a proof of semantic soundness for the full system. However, it seems clear that one could prove a syntactic soundness result for the full system, as we do for its extension with complements in the success semantics (Section 5) and for the constrained type system (Section 6). From syntactic soundness (progress and preservation), one also obtains that well-typed programs don't go wrong and that ill-typed programs don't evaluate, but we note that these statements only concern closed terms M , whereas semantic soundness concerns the truth of judgements in general.

It is perhaps unsurprising to note that the system is incomplete: there are true judgements that are not provable. For example, the term $\text{ifz } 0 \text{ then } \underline{1} \text{ else } \underline{\text{id}}$ is neither well-typed nor ill-typed. Semantically, this term evaluates to $\underline{1}$, but we cannot prove $\vdash \text{ifz } 0 \text{ then } \underline{1} \text{ else } \underline{\text{id}} : \text{Ok}$ because our rules for conditionals do not allow for a sufficiently precise case analysis. On a related note, one can observe that there are terms that are *both* well-typed and ill-typed: namely certain terms that diverge. The subject of the derivation at the end of Section 3 is one such example.

4.2 Further Discussion of the Semantics

One may wonder if the theory would be better developed using a semantics that has more symmetrical judgements, such as call-by-name (CBN). In a CBN semantics, in order for the standard (right) typing rules to be sound, we are forced to interpret $M : A$ on the left as ‘ M either evaluates to an A or diverges’ rather than, as in CBV, ‘ M evaluates to an A ’. Thus, the meaning of a formula in the premise and the meaning in the consequent become identical, which may seem desirable.

However, $M : A$ as a premise in CBN is a much weaker assumption, and reasoning on the left becomes very weak as a result. Consider a simple judgement of the form:

$$\text{ifz } M \text{ then succ}(x) \text{ else succ}(x) : \text{Nat} \vdash x : \text{Nat}$$

In CBV, this judgement is true independent of the choice of M , and is provable using the (IfZL2) rule – both branches require x to be a Nat, so necessarily the whole term requires x to be a Nat. The reason that this is sound depends crucially on knowing that the if-expression terminates. In CBV, $\text{ifz } M \text{ then succ}(x) \text{ else succ}(x) : \text{Nat}$ on the left means that we can assume the if-expression evaluated to a Nat (i.e. terminated), and hence one of the two branches evaluated to a Nat, and whichever branch that was, it used x at type Nat, and so the conclusion follows.

By contrast, in CBN, $\text{ifz } M \text{ then succ}(x) \text{ else succ}(x) : \text{Nat}$ on the left means that we can assume only that the if-expression either evaluated to a Nat or diverged. Therefore, we do not even know that execution of the guard M necessarily terminated. In the case that the guard M diverged, *neither* of the branches was ever executed. In this situation, it can be that no execution of the program actually uses x at type Nat, and so it is not true that $x : \text{Nat}$ necessarily follows. For example, the judgement $\text{ifz div then succ}(x) \text{ else succ}(x) : \text{Nat} \vdash x : \text{Nat}$ is *not* true in CBN, a counterexample is $x := \text{id}$. Therefore, the (IfZL2) rule for reasoning about cases on the left is unsound in CBN.

In fact, in CBN, when we assume $\text{ifz } M \text{ then } P \text{ else } Q : A$ on the left, we can conclude almost nothing about the necessary behaviour of P and Q ! This prevents us from reasoning about almost any interesting program. We do not know that either of P and Q is ever executed, so we can only use the behaviour of the guard M to justify some conclusion. The same is true of an application $MN : A$ on the left – we only know that MN either yielded an A or diverged and thus we cannot conclude anything about the necessary behaviour of N or $M[N/x]$, neither of which may have been executed if the execution of M did not terminate. Hence, (AppL) is unsound. To restore soundness it seems likely that one would have to equip the system with an ability to reason about termination explicitly. An approach like this was taken by Vazou et al. [2014] to repair a similar unsoundness when adapting the CBV *Liquid Types* system for Haskell.

5 COMPLEMENTS AND THE SUCCESS SEMANTICS

The two-sided type systems of this paper provide a form of negation at the meta level: to establish $M \notin \mathcal{T}[[A]]$ we can prove $M : A \vdash$. Since there are already a number of one-sided (traditional) type systems that incorporate a negation or complement operator on types, a natural question regards the extent to which two-sided reasoning can be simulated in a one-sided system with complements; in other words if one can internalise the set of judgements of shape $M : A \vdash$ by some type A^c .

Definition 5.1. We extend the grammar of types in Definition 2.3 by a complement operator, $A ::= \dots \mid A^c$, and we extend the system of rules with the two below.

Complement Rules

$$\begin{array}{c} \text{(CompL)} \quad \frac{\Gamma \vdash M : A, \Delta}{\Gamma, M : A^c \vdash \Delta} \qquad \text{(CompR)} \quad \frac{\Gamma, M : A \vdash \Delta}{\Gamma \vdash M : A^c, \Delta} \end{array}$$

The correspondence between these two rules and those for negation in sequent calculi supporting propositional logic should be evident. Intuitively, the semantics of the new operator is to take

the complement of the type with respect to the set of closed values. Rule (CompL), is closely related to our disjointness rule, and (Dis) can be derived from (CompL) if the system is augmented with a sufficiently strong subtyping theory, because $A \cap B = \emptyset$ iff $A \subseteq B^c$. This rule is sound for our call-by-value semantics, but rule (CompR) is *unsound*. The problem is that when $M : A^c$ is not satisfied on the right by some θ , it does not imply that $M : A$ will necessarily be satisfied by θ on the left.

For example, consider the derivation to the right. If this were sound in the call-by-value semantics, $\text{pred}(\text{id})$ would be guaranteed to *diverge*, since we would have that $\text{pred}(\text{id}) \in \mathcal{T}_\perp[\llbracket \text{Ok}^c \rrbracket]$, and $\llbracket \text{Ok}^c \rrbracket = \text{Vals}_0 \setminus \text{Vals}_0 = \emptyset$. This is clearly absurd, because $\text{pred}(\text{id})$ goes wrong. Thus our semantics of Section 4 does not support (CompR).

$$\begin{array}{c} \dots \\ \hline \vdash \text{id} : \text{Nat} \rightarrow \text{Nat} \\ \text{(Dis)} \quad \hline \text{id} : \text{Nat} \vdash \\ \text{(OkPL)} \quad \hline \text{pred}(\text{id}) : \text{Ok} \vdash \\ \text{(CompR)} \quad \hline \vdash \text{pred}(\text{id}) : \text{Ok}^c \end{array}$$

5.1 The Success Semantics

The key issue is that, if we internalise the judgements of shape $M : A \vdash$ by some type A^c , in the way that we have indicated, then we should expect that $M : A \vdash$ and $\vdash M : A^c$ have the same meaning. However, $M : A \vdash$ can hold of some term M that gets stuck (goes wrong); but this is impossible for M that satisfies $\vdash M : A^c$, because the semantics of the judgement ensures that any such term either diverges or evaluates to an A value. Thus, we have only the backward direction of the desired equivalence: $M \notin \mathcal{T}[\llbracket A \rrbracket]$ iff $M \in \mathcal{T}_\perp[\llbracket A^c \rrbracket]$.

Our approach will be to weaken the semantics of typing on the right of the judgement, so that a formula $M : A$ on the right *can* (always) be satisfied by a term M that goes wrong.

Definition 5.2. The *success semantics* is defined as follows. First, we define, for each type A , the set of closed terms that may go wrong, diverge, or evaluate to a value in $\llbracket A \rrbracket$.

$$\mathcal{T}_\perp[\llbracket A \rrbracket] = \{ M \mid M \in \mathcal{T}[\llbracket A \rrbracket] \text{ or } M \text{ diverges or } M \text{ goes wrong} \}$$

We redefine the meaning of the sufficiency arrow type to allow the body to go wrong when executing on an argument:

$$\llbracket A \rightarrow B \rrbracket = \{ \text{fix } f(x). M \in \text{Vals}_0 \mid \forall V \in \llbracket A \rrbracket. M[V/x][\text{fix } f(x). M/f] \in \mathcal{T}_\perp[\llbracket B \rrbracket] \}$$

Finally, we redefine satisfaction on the right by saying that a formula $M : A$ is *satisfied by θ on the right* just if $M\theta \in \mathcal{T}_\perp[\llbracket A \rrbracket]$.

The success semantics has the strong point that the judgements are now symmetrical in the sense of the above equivalence. Thus both (CompL) and (CompR) are sound for the success semantics. In fact, we will show that all the typing rules presented in Sections 2 and 3 are (syntactically) sound for the success semantics too.

A considerable disadvantage of the success semantics is that we lose the true negatives theorem *well-typed programs don't go wrong*. Now, proving $\vdash M : \text{Ok}$ means that M may either evaluate, diverge or go wrong. In other words, it means nothing at all! We do however retain *ill-typed programs don't evaluate*, so this system is exclusively for proving that programs behave badly. This puts this system in the same territory as Erlang's celebrated *success types* [Lindah and Sagonas 2006], which similarly provide no guarantees for terms that are well-typed.

However, the symmetry in the system allows for a considerable saving. Under the success semantics, $\llbracket A \multimap B \rrbracket = \llbracket A^c \rightarrow B^c \rrbracket$ – the conditions on membership in these types are the contrapositive of each other. Thus, there is the potential to simply define $A \multimap B$ as an abbreviation for $A^c \rightarrow B^c$. Then, the symmetry between (AppL) and (AppR), and the symmetry between (FixsR) and

(FixnR) can be exploited to derive necessity from sufficiency + complement:

$$\begin{array}{c}
 \text{(AppR)} \frac{\Gamma \vdash M : B \multimap A, \Delta}{\text{(CompL)} \frac{\Gamma \vdash MN : A^c, \Delta}{\Gamma, MN : A \vdash \Delta}} \quad \text{(CompR)} \frac{\Gamma, N : B \vdash \Delta}{\Gamma \vdash N : B^c, \Delta} \\
 \text{(CompL)} \frac{\Gamma, f : A \multimap B \vdash M : B^c, x : A, \Delta}{\Gamma, f : A \multimap B, x : A^c \vdash M : B^c, \Delta} \quad \text{(CompR)} \frac{\Gamma, f : B \multimap A, M : B \vdash x : A, \Delta}{\Gamma, f : A \multimap B \vdash M : B^c, x : A, \Delta} \\
 \text{(FixsR)} \frac{\Gamma \vdash \text{fix } f(x). M : A \multimap B, \Delta}{\Gamma \vdash \text{fix } f(x). M : A \multimap B, \Delta}
 \end{array}$$

Thus, in the success system, the sufficiency arrow in combination with complements provides a complete treatment of the necessity arrow, and we could dispense with (FixnR) and (AppL) if desirable.

5.2 The One-Sided System

We can go further by exploiting a key symmetry of the typing rules that we have presented so far.

Definition 5.3 (Variable typings and type environments.). Let us say that a typing formula $M : A$ is a *variable typing* just if M is a variable, and otherwise it is a *non-variable typing*. Let us say that an environment Γ is a *type environment* just in case every formula therein is a variable typing.

In each of the rules we have introduced in Sections 2 and 3, and so far in 5, observe the following. If the conclusion has at most one non-variable typing formula, then each of the hypotheses will have at most one non-variable typing formula too. For example, one can see by inspection of Examples 2.7 and 2.8 of Section 2 that, in each judgement of the respective proof trees, at most one typing formula is non-variable.

When a judgment contains at most one non-variable typing then, in the success semantics, it is equivalent to a judgement in which a non-variable typing is the only typing formula on the right-hand side of the turnstile. Thus, in the success type system, we are able to *normalise* the typing rules by using (CompL) and (CompR) to exchange the positions of formulas between the two sides, until they form a traditional, one-sided type system. In fact, the use of complements allows us to eliminate further redundancies, and so we obtain a smaller system.

Definition 5.4 (One-sided type system). A *judgement* of the one-sided system is a pair $\Gamma \vdash M : A$ of a type environment Γ and a typing $M : A$. Provability is defined by the rules of Figure 5. In rules (Let1), (Let2), (Let3), (Abs) we require that the bound variables are not mentioned in Γ .

In many cases, the rules of Figure 5 derive from their two-sided counterparts simply by normalising the judgements involved, possibly by introducing complements. Rule (Disj) arises from (Dis) this way. Normalisation often introduces a complemented typing $M : A^c$ in the conclusion, but in most cases it is sound to generalise to $M : A$. An example is deriving (IfZ1) from (IfZL1).

The rule (IfZ2) arises in this way, and it subsumes both (IfZR) and (IfZL2). However, the soundness of this one-sided rule is not obvious at first glance. It is almost the same as the right side rule (IfZR), except that (IfZR) has a third premise. In (IfZ2) this premise is absent, and we need only show that each branch has type A in order to conclude that the whole conditional has type A – *whether or not the guard is of type Nat*. However, this is *sound* for the success semantics, because if the guard normalises to something other than a numeral, then the conditional will go wrong, and thus satisfies any success type.

The rules (Ok), (OkC) and (Contra) represent structural features of the success semantics; namely that every term satisfies $\mathcal{T}_\perp \not\models [\text{Ok}]$, that no term satisfies $\mathcal{T} \models [\text{Ok}^c]$ and that $\mathcal{T} \models [A] \cap \mathcal{T} \models [A^c]$ is empty for every type A . Rule (Contra) is necessary in order to account for the possibility that a two-sided judgment is provable due to a contradiction among the variable typings. For example, the two-sided judgment $x : \text{Nat}, M : A \vdash x : \text{Nat}$ is clearly provable using (Id), but when normalised to a one-sided judgement, it becomes $x : \text{Nat}, x : \text{Nat}^c \vdash M : A$ and (Var) is not applicable.

$$\begin{array}{c}
\text{(Ok)} \frac{}{\Gamma \vdash M : \text{Ok}} \quad \text{(OkC1)} \frac{}{\Gamma, x : \text{Ok}^c \vdash M : A} \quad \text{(OkC2)} \frac{\Gamma \vdash M : \text{Ok}^c}{\Gamma \vdash M : A} \\
\\
\text{(Contra)} \frac{}{\Gamma, x : A, x : A^c \vdash M : A} \quad \text{(Var)} \frac{}{\Gamma, x : A \vdash x : A} \quad \text{(Disj)} \frac{\Gamma \vdash M : B}{\Gamma \vdash M : A^c} B \parallel A \\
\\
\text{(Zero)} \frac{}{\Gamma \vdash \text{zero} : \text{Nat}} \quad \text{(Succ1)} \frac{\Gamma \vdash M : \text{Nat}}{\Gamma \vdash \text{succ}(M) : \text{Nat}} \quad \text{(Succ2)} \frac{\Gamma \vdash M : \text{Nat}^c}{\Gamma \vdash \text{succ}(M) : A} \\
\\
\text{(Pred1)} \frac{\Gamma \vdash M : \text{Nat}}{\Gamma \vdash \text{pred}(M) : \text{Nat}} \quad \text{(Pred2)} \frac{\Gamma \vdash M : \text{Nat}^c}{\Gamma \vdash \text{pred}(M) : A} \\
\\
\text{(Fix)} \frac{\Gamma, f : A \rightarrow B, x : A \vdash M : B}{\Gamma \vdash \text{fix } f(x). M : A \rightarrow B} \quad \text{(Let3)} \frac{\Gamma \vdash N : A}{\Gamma \vdash \text{let } (x, y) = M \text{ in } N : A} \\
\\
\text{(Let2)} \frac{\Gamma \vdash M : (B_1 \times B_2)^c \quad \Gamma, x_i : B_i^c \vdash N : A \ (\forall i)}{\Gamma \vdash \text{let } (x_1, x_2) = M \text{ in } N : A} \quad \text{(Let1)} \frac{\Gamma \vdash M : B \times C \quad \Gamma, x_1 : B, x_2 : C \vdash N : A}{\Gamma \vdash \text{let } (x_1, x_2) = M \text{ in } N : A} \\
\\
\text{(App1)} \frac{\Gamma \vdash M : B \rightarrow A \quad \Gamma \vdash N : B}{\Gamma \vdash MN : A} \quad \text{(App2)} \frac{\Gamma \vdash M : (\text{Ok}^c \rightarrow A)^c}{\Gamma \vdash MN : A} \quad \text{(App3)} \frac{\Gamma \vdash N : \text{Ok}^c}{\Gamma \vdash MN : A} \\
\\
\text{(Pair1)} \frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B}{\Gamma \vdash (M, N) : A \times B} \quad \text{(Pair2)} \frac{\Gamma \vdash M_i : \text{Ok}^c}{\Gamma \vdash (M_1, M_2) : A} \quad \text{(Pair3)} \frac{\Gamma \vdash M_i : A_i^c}{\Gamma \vdash (M_1, M_2) : (A_1 \times A_2)^c} \\
\\
\text{(IfZ1)} \frac{\Gamma \vdash M : \text{Nat}^c}{\Gamma \vdash \text{ifz } M \text{ then } N \text{ else } P : A} \quad \text{(IfZ2)} \frac{\Gamma \vdash N : A \quad \Gamma \vdash P : A}{\Gamma \vdash \text{ifz } M \text{ then } N \text{ else } P : A}
\end{array}$$

Fig. 5. One-Sided Type Assignment

THEOREM 5.5. *Suppose Γ and Δ are type environments.*

- *If $\Gamma, M : A \vdash \Delta$ in the two-sided system, then $\Gamma \cup \Delta^c \vdash M : A^c$ in the one-sided system.*
- *If $\Gamma \vdash M : A, \Delta$ in the two-sided system, then $\Gamma \cup \Delta^c \vdash M : A$ in the one-sided system.*

Example 5.6. Consider again the judgement $\vdash \text{twice} : (A \multimap A) \rightarrow A \multimap A$, which we proved as Example 2.8 of Section 2. As in that example, we use Γ to abbreviate $\{f : A \multimap A\}$:

$$\begin{array}{c}
\text{(Var)} \frac{}{\Gamma, x : A^c \vdash f : A \multimap A} \quad \text{(Var)} \frac{}{\Gamma, x : A^c \vdash x : A^c} \\
\text{(App)} \frac{}{\Gamma, x : A^c \vdash f x : A^c} \\
\text{(Abs)} \frac{\Gamma, x : A^c \vdash f(f x) : A^c}{\Gamma \vdash \lambda x. f(f x) : A \multimap A} \\
\text{(Abs)} \frac{}{\vdash \lambda f x. f(f x) : (A \multimap A) \rightarrow A \multimap A}
\end{array}$$

We prove syntactic soundness for the one-sided system using a progress and preservation argument. In fact, progress as usually understood is not necessary – since we allow going wrong

on the right, there is no requirement to show that a subject in normal form is a value, merely that values have sensible types. We get the syntactic soundness of the two-sided system under the success semantics as a corollary via Theorem 5.5.

Let us say that a closed term M is *ill-typed* in the one-sided system just if $\vdash M : \text{Ok}^c$.

THEOREM 5.7 (ONE-SIDED SYNTACTIC SOUNDNESS). *If M is ill-typed, then M does not evaluate.*

6 A CONSTRAINED TYPE SYSTEM

We now depart from the study of our PCF-like language, to introduce a two-sided *constrained* type system. Our aim is to illustrate how one can take the core of an existing one-sided system, which is useful in practice, and extend it to a two-sided system, whilst retaining its good properties (such as its true negatives theorem⁶ and computable type inference).

Starting from the two-sided rules for pure terms with fixpoints of Figure 1, we add type constraints, top-level definitions, term constants for algebraic datatypes and pattern-matching, and types corresponding to datatype constructors. This gives us a two-sided system whose features are similar to those of Aiken et al. [1994]; Jones and Ramsay [2021]; Marlow and Wadler [1997], which are capable of precise reasoning about datatypes and matching.

Definition 6.1. We assume a denumerable set of *top-level identifiers*, written typically as f, g , and a denumerable set of *local variables*, typically f, g, x, y, z . We also fix a finite signature C of ranked datatype *constructors*, typically c . We will always assume that the binary pair constructor, written mixfix as $(,)$ is in the signature. We consider various kinds of program expressions:

$$\begin{aligned} p, q &::= c(x_1, \dots, x_m) \\ M, N, P, Q &::= x \mid c(M_1, \dots, M_m) \mid MN \mid \text{fix } f(x).M \mid \text{match } M \text{ with } \{|_{i=1}^k (p_i \mapsto P_i)\} \\ V, W &::= x \mid c(V_1, \dots, V_n) \\ \mathcal{M} &::= \epsilon \mid f = M; \mathcal{M} \end{aligned}$$

As usual, we identify terms up to renaming of bound variables. We consider a term to be *closed* just if it contains no free variables (but it may contain top-level function identifiers). As before, we consider $\lambda x. M$ as an abbreviation for $\text{fix } f(x). M$ when f is not free in M .

We make the following additional requirements: (i) in a pattern $c(x_1, \dots, x_n)$ the arity of c is n and the x_i are pairwise distinct, (ii) in a pattern-match term $\text{match } M \text{ with } \{|_{i=1}^k (p_i \mapsto P_i)\}$, the family $(p_i)_{i=1}^k$ is orthogonal (i.e. for every distinct i and j , p_i and p_j are headed by distinct constructors) and *all* bound variables throughout the family have distinct names, (iii) in module $f_1 = M_1; \dots; f_n = M_n$, each f_i is distinct (there are not two definitions for the same top-level identifier).

Thus, we will feel free to index pattern families by the set of constructors, say I , that head their alternatives $|_{c \in I} c(\overline{x_c})$, and sometimes treat a module $\mathcal{M}$ as a partial map from identifiers to terms.

The first category are the *patterns*, which are required to be *simple* in the sense of being shallow and constructor headed. The second category are the *terms*. As in our PCF-like language, we have an applied λ -calculus with fixpoints, but this time we are interested in datatype constructors and pattern matching and thus we have constants accordingly. Thirdly, we have *modules* $\mathcal{M}$, which are just sequences of definitions for top-level identifiers. Finally, *values* are either variables, fixpoint abstractions or datatype constructor terms built entirely from values.

Remark 6.1. The requirement for simple matching is to ease the presentation. We could allow arbitrary patterns at the expense of more complex subtype and consistency checking. Since this is not the focus of our work and is anyway well-covered elsewhere, we will be content with the simple case. Note also that the rank (arity) of a constructor is built into the syntax, so we will never

⁶For this reason, we will use a standard CBV semantics and not the success semantics of the previous section.

consider the possibility of supplying a constructor with an inappropriate number of arguments since it is not considered a well-formed term.

Example 6.2. Let us assume a constructor signature C , which, in addition to the binary pair constructor $(\ , \)$, contains the binary list constructor ‘cons’, written infix $::$, and the nullary empty list constructor ‘nil’, written $[]$. We can define the head and map functions by:

$$\begin{aligned} \text{head} &= \lambda xs. \text{match } xs \text{ with } \{y :: ys \mapsto y\} \\ \text{map} &= \text{fix } m(f). \lambda xs. \text{match } xs \text{ with } \{[] \mapsto [] \mid y :: ys \mapsto f\ y :: m\ f\ ys\} \end{aligned}$$

As before, we adopt a call-by-value reduction strategy and reuse the notation⁷ $M \triangleright N$. The sequence of arguments to a datatype constructor is evaluated from left to right. Due to the requirement for shallow and orthogonal patterns, matching can be executed by simply indexing into the pattern family. A full definition is in the long version of the paper [Ramsay and Walpole 2023].

6.1 Types and Type Constraints

Our type system is designed for reasoning about the shape of datatype constructions. It is a *constrained type system* (see e.g. [Odersky et al. 1999]), so each judgement is parametrised by a set of type constraints C , which restrict the possible instantiations of type variables.

Our types are a cut-down version of the types defined in the constrained type system of Aiken et al. [1994], in that for each datatype constructor c of arity n , we can form a corresponding constructor type $c(A_1, \dots, A_n)$. Intuitively, this type represents the set of all values of shape $c(V_1, \dots, V_n)$, where each V_i has type A_i . We allow for universal polymorphism through the construction of *constrained type schemes*, $\forall \vec{a}. C \Rightarrow A$. Intuitively, a top-level function that has a such a scheme is guaranteed to behaves like $A[\vec{B}/\vec{a}]$ for each instantiation $\vec{B}$ of the type variables $\vec{a}$ that satisfy the constraints C .

Definition 6.3. We assume a denumerable collection of *type variables* a, b and so on. The types are stratified as follows:

$$\begin{aligned} (\text{Sum Types}) \quad K &::= \Sigma_{i=1}^k c_i(\vec{A}_i) \\ (\text{Monotypes}) \quad A, B &::= a \mid K \mid \text{Ok} \mid A \rightarrow B \mid A \multimap B \\ (\text{Type Schemes}) \quad S &::= A \mid \forall \vec{a}. C \Rightarrow A \end{aligned}$$

Here C (and sometimes D) is a finite set of *type constraints* (also *subtype formulas*), each of shape $A \sqsubseteq B$ (i.e. between monotypes). We write $A \equiv B$ as an abbreviation for the two constraints $A \sqsubseteq B$ and $B \sqsubseteq A$. We identify type schemes up to the renaming of bound type variables and we assume that arrows associate to the right. We require that type schemes are *closed* in the sense that they have no free type variables.

We consider the sum type $\Sigma_{i=1}^k c_i(\vec{A}_i)$ as a finite set $\{c_1(\vec{A}_1), \dots, c_k(\vec{A}_k)\}$, and we require that the elements are orthogonal in the sense that $c_i = c_j$ implies $i = j$. See also Remark 6.1. In examples, we will often write a particular sum type $\Sigma_{i=1}^3 c_i(\vec{A}_i)$ as a list of summands $c_1(\vec{A}_1) + c_2(\vec{A}_2) + c_3(\vec{A}_3)$. In particular, when the sum is a singleton $\Sigma_{i=1}^1 c_i(\vec{A}_i)$, which is quite typical, we just write $c_1(\vec{A}_1)$.

We don't have an explicit notion of recursive types with which to type recursively defined data structures. However, as is well known, our constructor types together with (recursive) type constraints can capture the same notion implicitly. For example, our map function from Example 6.2 can be assigned the type:

$$\begin{aligned} \text{map} : \forall a\ b\ \ell_a\ \ell_b. C \Rightarrow (a \rightarrow b) \rightarrow \ell_a \rightarrow \ell_b \\ \text{where } C = \{ \ell_a \equiv [] + (a :: \ell_a), \ell_b \equiv [] + (b :: \ell_b) \} \end{aligned}$$

⁷Actually the one-step relation is parametrised by a module M , but we leave this implicit.

$$\begin{array}{c}
\text{(IdS)} \frac{}{C, A \sqsubseteq B \vdash A \sqsubseteq B} \quad \text{(TrS)} \frac{C \vdash A_1 \sqsubseteq A_2 \quad C \vdash A_2 \sqsubseteq A_3}{C \vdash A_1 \sqsubseteq A_3} \\
\\
\text{(ToS)} \frac{C \vdash A' \sqsubseteq A \quad C \vdash B \sqsubseteq B'}{C \vdash A \rightarrow B \sqsubseteq A' \rightarrow B'} \quad \text{(FrS)} \frac{C \vdash A \sqsubseteq A' \quad C \vdash B' \sqsubseteq B}{C \vdash A \multimap B \sqsubseteq A' \multimap B'} \\
\\
\text{(OkS)} \frac{}{C \vdash A \sqsubseteq \text{Ok}} \quad \text{(SmS)} \frac{C \vdash A_{i,c} \sqsubseteq B_{i,c} \ (\forall c \in I, \forall i \in [1..n_c])}{C \vdash \sum_{c \in I} c(A_{1,c}, \dots, A_{n_c,c}) \sqsubseteq \sum_{d \in J} d(B_{1,d}, \dots, B_{n_b,b})} I \sqsubseteq J
\end{array}$$

Fig. 6. Constrained subtyping.

This type says that `map` takes a function from a to b and a list of a and returns a list of b . Intuitively, the type ‘list of a ’ is described by the type variable ℓ_a under the constraint that $\ell_a \equiv [] + (a :: \ell_a)$ and similarly for ‘list of b ’ with ℓ_b constrained so that $\ell_b \equiv [] + (b :: \ell_b)$.

In a constrained type system, it is usual to define subtyping with respect to a context containing subtyping formulas, and so we have the following.

Definition 6.4. A *subtyping judgement* is a triple $C \vdash A \sqsubseteq B$ in which C is a set of type constraints and $A \sqsubseteq B$ is a type constraint. Provability is defined using the rules of Figure 6. We extend the notion of provability to sets of constraints, writing $C \vdash C'$ just if $C \vdash A \sqsubseteq B$ for every $A \sqsubseteq B \in C'$.

The rule (IdS) allows for justification with respect to the context, and the rule (TrS) ensures closure under transitivity of subtyping. Rule (ToS) gives the familiar relationship between sufficiency arrow types and (FrS) describes the dual relationship between necessity arrow types. Rule (OkS) puts `Ok` at the top of the subtype ordering. Finally, the rule (SmS) allows for subtyping between sum types. It says that a sum type K_1 is a subtype of K_2 just if whenever $c(A_1, \dots, A_n)$ is a summand of K_1 , then there is a summand of shape $c(B_1, \dots, B_n)$ in K_2 and, moreover, the argument types are covariantly related. For example, using the constructors from Example 6.2, we have $\vdash [] \sqsubseteq [] + (\text{Ok} :: \text{Ok})$ and $a \sqsubseteq b \vdash (a :: []) \sqsubseteq [] + (b :: [])$.

6.2 Type Assignment

Finally, we have the two-sided type system itself. In the interests of making the syntactical soundness proof and type inference as straightforward as possible (by making the system as close to a traditional type system as possible), we present the type system as an *intuitionistic* sequent calculus, in the sense that there will be allowed *at most one* formula on the right hand side. We choose not to have an explicit component in the judgement for constraints in order to simplify the notation.

Definition 6.5 (Typing Formulas and Type Assignment). A *typing formula* is a pair of shape either $M : A$ or $f : S$, with M a term, A a monotype, f a top-level identifier and S a type scheme. A *typing judgment* of the system consists of a pair $\Gamma \vdash \Delta$ in which Γ is a finite set of typing and subtype formulas and Δ is a finite set of typing formulas of shape $M : A$ and whose size is at most 1. For brevity, by some abuse, we will write Γ even for the subset $\{A \sqsubseteq B \mid A \sqsubseteq B \in \Gamma\}$ of subtype constraints contained therein.

The rules of the type system are given in Figure 7. We additionally require that: (i) in the rules (FixsR), (FixnR), (MchL), (MchR) and (FixR), the bound variables in the principal subject of the conclusion do not occur in Γ or Δ ; and (ii) in the rule (GVar), the vector of types $\vec{B}$ has the same length as the vector of type variables $\vec{a}$; and (iii) the rules (CnsK) and (CnsL) require that $n > 0$ and $1 \leq i \leq n$.

Structural	
(VarK) $\frac{}{\Gamma \vdash x : \text{Ok}}$	(GVar) $\frac{}{\Gamma, f : \forall \vec{a}. C \Rightarrow A \vdash f : A[\vec{B}/\vec{a}]} \Gamma \vdash C[\vec{B}/\vec{a}]$
(Id) $\frac{}{\Gamma, x : A \vdash x : A}$	(SubL) $\frac{\Gamma, M : B \vdash \Delta}{\Gamma, M : A \vdash \Delta} \Gamma \vdash A \sqsubseteq B$ (SubR) $\frac{\Gamma \vdash M : B}{\Gamma \vdash M : A} \Gamma \vdash B \sqsubseteq A$
Functions	
(FixsR) $\frac{\Gamma, f : A \rightarrow B, x : A \vdash M : B}{\Gamma \vdash \text{fix } f(x). M : A \rightarrow B}$	(FixnR) $\frac{\Gamma, f : A \multimap B, M : B \vdash x : A}{\Gamma \vdash \text{fix } f(x). M : A \multimap B}$
(AppL) $\frac{\Gamma \vdash M : B \multimap A \quad \Gamma, N : B \vdash \Delta}{\Gamma, MN : A \vdash \Delta}$	(AppR) $\frac{\Gamma \vdash M : B \rightarrow A \quad \Gamma \vdash N : B}{\Gamma \vdash MN : A}$
Constructors	
(CnsL) $\frac{\Gamma, M_i : A_i \vdash \Delta}{\Gamma, c(M_1, \dots, M_m) : c(A_1, \dots, A_m) + K \vdash \Delta}$	(CnsR) $\frac{\Gamma \vdash M_i : A_i (\forall i)}{\Gamma \vdash c(M_1, \dots, M_m) : c(A_1, \dots, A_m)}$
Matching	
(MchR) $\frac{\Gamma \vdash M : \Sigma_{i=1}^k p_i [\vec{B}_x/x] \quad \Gamma \cup \{x : B_x \mid x \in \text{FV}(p_i)\} \vdash P_i : A (\forall i)}{\Gamma \vdash \text{match } M \text{ with } \{ _{i=1}^k (p_i \mapsto P_i)\} : A}$	(MchL) $\frac{\Gamma, P_i : A \vdash x : B_x (\forall i. \forall x \in \text{FV}(p_i)) \quad \Gamma, (M, P_i) : (p_i [\vec{B}_x/x], A) \vdash \Delta (\forall i)}{\Gamma, \text{match } M \text{ with } \{ _{i=1}^k (p_i \mapsto P_i)\} : A \vdash \Delta}$
Evaluation	
(CnsK) $\frac{\Gamma, M_i : \text{Ok} \vdash \Delta}{\Gamma, c(M_1, \dots, M_n) : \text{Ok} \vdash \Delta}$	(Funk) $\frac{\Gamma, M : \text{Ok} \multimap A \vdash \Delta}{\Gamma, MN : \text{Ok} \vdash \Delta}$
Disjointness	
(CnsDL) $\frac{}{\Gamma, c(M_1, \dots, M_n) : A \vdash \Delta} \begin{array}{l} A \text{ an arrow, or shape} \\ \Sigma_{d \in I} d(\dots) \text{ with } c \notin I \end{array}$	(FixDL) $\frac{}{\Gamma, \text{fix } f(x). M : K \vdash \Delta}$

Fig. 7. Constrained type assignment.

Many of the typing rules of Figure 7 are recognisable from Sections 2 and 3, so we will just comment on new aspects. First, we have separate typing rules for local variables (introduced by abstractions and pattern-match cases) (Id), and top-level identifiers (GVar). Top-level identifiers can be assumed to have polymorphic type schemes $\forall \vec{a}. C \Rightarrow A$, so the (GVar) rule allows for the instantiation of quantified type variables $\vec{a}$ by a vector of monotypes $\vec{B}$, subject to the requirement that each of the constraints in C is already derivable from the type constraints assumed in Γ . The (CnsL), (CnsR) and (CnsK) rules allow for refuting and affirming the types of constructor-headed terms. Note that the (CnsR) rule allows for concluding the singleton sum $c(A_1, \dots, A_m)$, whereas the (CnsL) rule allows a more general sum of shape $c(A_1, \dots, A_m) + K$. This is because the former may yet be generalised to the larger type $c(A_1, \dots, A_m) + K$ using (SubR) (which would be impossible on the left, since (SubL) only allows for concluding smaller types from larger ones).

The rules (MchL) and (MchR) are used for typing pattern-matching on the left and right respectively. The (MchR) rule is relatively standard: one must choose a typing B_x for each pattern-bound variable x (recall from additional requirement (ii) of Definition 6.1, that we assume all bound variables throughout the pattern-match term to have distinct names), in such a way that the scrutinee M is inside the sum of the induced pattern types. These are obtained by taking each pattern case $c(x_1, \dots, x_n)$ and replacing the free variables x_i by the corresponding type B_{x_i} , giving $c(B_{x_1}, \dots, B_{x_n})$. Then one must show that every branch can guarantee an A .

In (MchL), one must first derive a necessary type B_x for each pattern-bound variable x , based on how it is used in the corresponding branch of the match. These types then give rise to a type for the scrutinee, as above. Then one must show that, for each branch i , either the desired conclusion Δ follows from the fact that the body of the branch P_i has type A , or already from the fact that the scrutinee has type $p_i[\overline{B_x/x}]$. This disjunction is encoded by asserting that the pair (M, P_i) has type $(p_i[\overline{B_x/x}], A)$ so as to maintain the invariant that at most one non-trivial typing is introduced in each premise. An example of how this is used is at the end of this subsection.

Instead of a single disjointness rule (Dis), in this system we have disjointness rules (CnsDL) and (FixDL) specific to each introduction form (see Section 2.2 for further discussion of these alternatives). We make this choice here to simplify type inference: we avoid the non-syntax directed rule (Dis), and we avoid the need to infer disjointness constraints in addition to type constraints.

Definition 6.6 (Top-level Function Typing). Top-level functions are typed according to the rule:

$$\frac{\Gamma \cup C \vdash M(f) : A}{\Gamma \vdash f : \forall \vec{a}. C \Rightarrow A} \vec{a} = \text{FV}(C) \cup \text{FV}(A)$$

We write $\vdash M : \Gamma$ just if, (i) every top-level function of M appears as a subject of Γ , and (ii) every top-level function typing $f : S \in \Gamma$ is properly justified $\Gamma \vdash f : S$.

Since each rule of the system allows for an arbitrary context Γ on the left, it satisfies weakening on the left. Consequently, rules (AbsR) and (AbnR) of Section 2 are admissible. Consider the head function of Example 6.2. We show that head requires a cons with an element of type a as input in order to produce an a , i.e. $\text{head} : \forall a. (a :: \text{Ok}) \multimap a$. The derivation starts:

$$\text{(AbnR)} \frac{\text{match } xs \text{ with } \{y :: ys \mapsto y\} : a \vdash xs : (a :: \text{Ok})}{\vdash \lambda xs. \text{match } xs \text{ with } \{y :: ys \mapsto y\} : (a :: \text{Ok}) \multimap a}$$

Then, according to (MchL) we must derive types for the bound variables y and ys based on how they were necessarily used to obtain type a in their branch. In this case, their branch body is just y and so we can assign y the type a and ys , which is not used in the branch, must be given Ok . Thus the scrutinee must have type $a :: \text{Ok}$. According to (MchL), we must show that the conclusion,

$xs : (a :: \text{Ok})$, either follows from this or from the body of the branch. In this case, it is clear that it follows already from the type of the scrutinee. This reasoning is captured as:

$$\frac{\text{(Id)} \frac{}{y : a \vdash y : a} \quad \text{(VarK)} \frac{}{y : a \vdash ys : \text{Ok}} \quad \text{(Cnsl)} \frac{\text{(Id)} \frac{}{xs : (a :: \text{Ok}) \vdash xs : (a :: \text{Ok})}}{(xs, y) : (a :: \text{Ok}, a) \vdash xs : (a :: \text{Ok})}}{\text{(MchL)} \frac{}{\text{match } xs \text{ with } \{y :: ys \mapsto y\} : a \vdash xs : (a :: \text{Ok})}}$$

Consider the map function from Example 6.2. We will show that, when map is given a function that requires an a to produce a b , then to produce a list of b requires it be given a list of a :

$$\text{map} : \forall a b \ell_a \ell_b. C \Rightarrow (a \multimap b) \rightarrow \ell_a \multimap \ell_b \\ \text{where } \{ [] + (a :: \ell_a) \sqsubseteq \ell_a, \ell_b \sqsubseteq [] + (b :: \ell_b) \}$$

The derivation begins as we have seen previously using (FixsR) and (AbnR). The key part is to show that, under $\Gamma = C \cup \{f : a \multimap b, m : (a \multimap b) \rightarrow \ell_a \multimap \ell_b\}$:

$$\Gamma, \text{match } xs \text{ with } \{[] \mapsto [] \mid y :: ys \mapsto f y :: m f ys\} : \ell_b \vdash xs : \ell_a$$

To use the (MchL) rule, we first derive types that were necessary for y and ys to produce a value of type ℓ_b in the cons branch (we omit some standard right-side reasoning in the latter).

$$\frac{\text{(Id)} \frac{}{\Gamma \vdash f : a \multimap b} \quad \text{(Id)} \frac{}{\Gamma, y : a \vdash y : a}}{\text{(AppL)} \frac{}{\Gamma, f y : b \vdash y : a}} \quad \frac{\dots \quad \text{(Id)} \frac{}{\Gamma, ys : \ell_a \vdash ys : \ell_a}}{\text{(AppL)} \frac{}{\Gamma, m f ys : \ell_b \vdash ys : \ell_a}} \\ \frac{\text{(Cnsl)} \frac{}{\Gamma, (f y :: m f ys) : [] + (b :: \ell_b) \vdash y : a}}{\text{(SubL)} \frac{}{\Gamma, (f y :: m f ys) : \ell_b \vdash y : a}} \quad \frac{\text{(Cnsl)} \frac{}{\Gamma, (f y :: m f ys) : [] + (b :: \ell_b) \vdash y : \ell_a}}{\text{(SubL)} \frac{}{\Gamma, (f y :: m f ys) : \ell_b \vdash y : \ell_a}}$$

Then, we show that $xs : \ell_a$ is a necessary consequence of each branch being of type ℓ_b . In fact, it follows directly from the induced type of the scrutinee in each (we omit the (Cnsl) at the root).

$$\frac{\text{(Id)} \frac{}{\Gamma, xs : (a :: \ell_a) \vdash xs : (a :: \ell_a)}}{\text{(SubR)} \frac{}{\Gamma, xs : (a :: \ell_a) \vdash xs : \ell_a}} \quad \frac{\text{(Id)} \frac{}{\Gamma, xs : [] \vdash xs : []}}{\text{(SubR)} \frac{}{\Gamma, xs : [] \vdash xs : \ell_a}}$$

As one final example, we show that map, given a function that requires an a to obtain a b , returns a cons with element of type b only if given a cons with element of type a , i.e:

$$\text{map} : \forall a b. (a \multimap b) \rightarrow (a :: \text{Ok}) \multimap (b :: \text{Ok})$$

The key part of the derivation is again the (MchL) rule. Under $\Gamma = \{f : a \multimap b, m : (a \multimap b) \rightarrow (a :: \text{Ok}) \multimap (b :: \text{Ok})\}$, we find, as above, that the cons case requires y to be an a (left), but we don't require anything special of ys (right):

$$\frac{\dots}{\Gamma, (f y :: m f ys) : \ell_b \vdash y : a} \quad \text{(VarK)} \frac{}{\Gamma, (f y :: m f ys) : \ell_b \vdash ys : \text{Ok}}$$

When showing that the desired conclusion follows from the two cases, we need to use the choice offered by the pair $(M, P_i) : (p_i[B_x/x], A)$ on the left of the second family of premises of (MchL). By using (Cnsl) to choose one component of the pair with which to continue the proof, we can effectively ignore irrelevant cases. Here, the nil case is excluded by the type $b :: \text{Ok}$ of the output (left), and the result follows from the type of the scrutinee in the cons case (abbreviating $M = f y :: m f ys$ and $\Delta = \{xs : (a :: \text{Ok})\}$):

$$\frac{\text{(Cnsl)} \frac{}{\Gamma, [] : (b :: \text{Ok}) \vdash \Delta}}{\text{(Cnsl)} \frac{}{\Gamma, (xs, []) : ([], b :: \text{Ok}) \vdash \Delta}} \quad \frac{\text{(Id)} \frac{}{\Gamma, xs : (a :: \text{Ok}) \vdash \Delta}}{\text{(Cnsl)} \frac{}{\Gamma, (xs, M) : (a :: \text{Ok}, b :: \text{Ok}) \vdash \Delta}}$$

We can now prove the example of ill-typedness in the introduction, the term that takes the head of a list obtained by mapping over the empty list will not evaluate: $\text{head}(\text{map}(\lambda x. x) []) : \text{Ok} \vdash$.

6.3 Constrained Type Inference

For the purpose of inferring types and, in the next part, proving soundness, we are going to restrict ourselves to judgements that are as close to those in a traditional type system as possible: they will have at most one typing whose subject is not a variable. The following definition is a variation of Definition 5.3 suitable for our constrained type system.

Definition 6.7 (Type Environments). A finite set of formulas Γ is said to be a *type environment* just if (i) all the typing formulas contained therein are variable typings $x : A$ and top-level identifier typings $f : A$, and (ii) if $M : A \in \Gamma$ and $M : B \in \Gamma$ then $A = B$. A type environment is said to be a *top-level type environment* just if all the typing formulas are top-level identifier typings.

Inference for the two-sided constrained type system works, broadly, like Hindley-Milner constrained type inference (see e.g. [Odersky et al. 1999]), but with two notable differences. The first is that most terms will have multiple ‘principal’ types. This is because there are several shapes of proof tree with the same subject in the conclusion, but whose inferred types are not necessarily related. Hence, type inference infers sets of types⁸. The other main difference is that our algorithm requires both the left environment Γ and the right environment Δ as input when inferring the types of a term on the left. Hence, inference is split into two procedures, InferL and InferR :

$$\text{InferL} \in \mathcal{P}(\text{Variable} \times \text{Type}) \rightarrow \text{Term} \rightarrow \mathcal{P}(\text{Variable} \times \text{Type}) \rightarrow \mathcal{P}(\text{Judgement})$$

$$\text{InferR} \in \mathcal{P}(\text{Variable} \times \text{Type}) \rightarrow \text{Term} \rightarrow \mathcal{P}(\text{Judgement})$$

The call $\text{InferL}(\Gamma)(M)(\Delta)$ returns a finite set of judgements⁹ $C \cup \Gamma, M : A \vdash \Delta$ that are principal in the sense that all provable judgements with the same left-environment Γ , subject M and right-environment Δ arise as a type-substitution instance of one of those in the set, but with a possibly stronger set of assumed type constraints. The call $\text{InferR}(\Gamma)(M)$ acts similarly on the right.

The algorithms terminate because, modulo subtyping, every proof tree has a maximum height that is determined by the shape of the term. As usual, subtyping can be absorbed into the other rules in a so-called algorithmic system, and then inference consists of (implicitly) constructing the finite set of principal proof trees for the given term and environment(s). Inferring on the right for the term $\lambda x. f x$ in the environment $\{f : [] \multimap \text{Ok}\}$, we will obtain:

$$\begin{array}{l} \text{Ok} \sqsubseteq a_1, a_1 \multimap a_2 \sqsubseteq a_3, f : [] \multimap \text{Ok} \vdash \lambda x. f x : a_3 \\ \text{Ok} \sqsubseteq a_1, [] \multimap \text{Ok} \sqsubseteq a_3, a_3 \sqsubseteq a_2 \multimap a_1, a_1 \multimap a_4 \sqsubseteq a_5, f : [] \multimap \text{Ok} \vdash \lambda x. f x : a_5 \\ a_2 \sqsubseteq a_1, [] \multimap \text{Ok} \sqsubseteq a_3, a_3 \sqsubseteq a_2 \multimap a_4, a_1 \multimap a_4 \sqsubseteq a_5, f : [] \multimap \text{Ok} \vdash \lambda x. f x : a_5 \\ a_1 \sqsubseteq a_2, [] \multimap \text{Ok} \sqsubseteq a_3, a_3 \sqsubseteq a_2 \rightarrow a_4, a_1 \rightarrow a_4 \sqsubseteq a_5, f : [] \multimap \text{Ok} \vdash \lambda x. f x : a_5 \end{array}$$

The first three correspond, modulo subtyping inferences, to proof trees rooted at (AbnR). The first corresponds to the case when the tree is then immediately closed by the (VarK) axiom. The second corresponds to proceeding by (AppL) before closing with (GVar) in the left branch and (VarK) in the right branch. The third is similar but corresponds to closing the right branch with (Id) instead. The fourth corresponds to a tree rooted at (AbsR), after which the shape is completely determined (modulo subtyping). However, notice that the constraints returned in this last case are inconsistent, we have $[] \multimap \text{Ok} \sqsubseteq a_3$ and $a_3 \sqsubseteq a_2 \rightarrow a_4$, but the former function type contains functions that go wrong after being given an input, whereas the latter function type does not. Indeed, one cannot

⁸One can observe the same phenomenon in, e.g. intersection type systems.

⁹We formalise it this way for convenience, but in practice it need only return C and A .

assign a sufficiency arrow type to this term in the given environment under consistent subtyping assumptions. The full definition is given in the long version [Ramsay and Walpole 2023].

THEOREM 6.8 (CORRECTNESS OF THE TYPE INFERENCE ALGORITHM). *Let Γ, Δ be type environments without type constraints, C and C' be sets of constraints, M be a term, and A, A' be types. Then:*

(Left Soundness) *If $(C \cup \Gamma, M : A \vdash \Delta) \in \text{InferL}(\Gamma)(M)(\Delta)$, then $C \cup \Gamma, M : A \vdash \Delta$ is provable.*

(Left Completeness) *If $C' \cup \Gamma, M : A \vdash \Delta$ is provable, then there exists a type substitution σ and a judgement $(C \cup \Gamma, M : A' \vdash \Delta) \in \text{InferL}(\Gamma)(M)(\Delta)$ such that $A = A'\sigma$ and $C' \vdash C\sigma$.*

(Right Soundness) *If $(C \cup \Gamma \vdash M : A) \in \text{InferR}(\Gamma)(M)$, then $C \cup \Gamma \vdash M : A$ is provable.*

(Right Completeness) *If $C' \cup \Gamma \vdash M : A$ is provable, then there exists a type substitution σ and a judgement $(C \cup \Gamma \vdash M : A') \in \text{InferR}(\Gamma)(M)$ such that $A = A'\sigma$ and $C' \vdash C\sigma$.*

6.4 Syntactic Soundness

Rather than proving semantic soundness, we take the opportunity to show how one can prove a syntactic soundness result in the sense of Wright and Felleisen [1994], but first we need to generalise the definition of well-typed and ill-typed to account for top-level identifiers.

Definition 6.9. Suppose M is closed and Γ is a consistent top-level type environment with $\vdash \mathcal{M} : \Gamma$.

- We say that M is *well-typed* in Γ just if $\Gamma \vdash M : \text{Ok}$.
- We say that M is *ill-typed* in Γ just if $\Gamma, M : \text{Ok} \vdash$.

The force of the qualifier *consistent* is to require that the type constraints in a type environment are not contradictory. Several definitions are possible, and we use an adaptation of the syntactical definition given by Eifrig et al. [1995]. Since it is orthogonal to the two-sided aspect of the work, the definition has been relegated to the long version of this paper [Ramsay and Walpole 2023].

An appropriate formulation of *progress* for two-sided systems must take into account a non-trivial subject on the left of the turnstile as well as the right. On the right, as usual, we have that terms can either make a step or are already values. However, in a typing $M : A$ on the left, it is possible that M can make a step, already be stuck or even be a value. However, in the latter case, the value must not be in A . We can state this succinctly as follows:

THEOREM 6.10 (PROGRESS). *Suppose M is closed and Γ is a consistent, top-level type environment.*

- *If $\Gamma \vdash M : A$ then either M can make a step, or M is a value*
- *If $\Gamma, M : A \vdash$ then either M can make a step, or $\Gamma \not\vdash M : A$*

The formulation uses a fact about typing in our two-sided systems that is peculiar to systems with the necessity arrow: all closed values are typable on the right. We already remarked in Section 3 that every (fixpoint) abstraction is typable on the right with $\text{Ok} \multimap A$, and a simple induction shows that every constructor-headed term has a corresponding constructor-headed type. Thus, $\Gamma \not\vdash M : A$ above in particular implies that M is not a value of type A .

For *preservation*, it is typical to prove some preliminary lemmas that show closure under well-typed substitutions. In the two-sided case, there are many more of these substitution lemmas, since one has to take account of the possibilities that the substituted-for variable occurs on the opposite side of the turnstile to the subject (as usual), on the same side, or not at all (which also relies on the fact that closed values are typable). However, preservation and then soundness follow directly.

THEOREM 6.11 (PRESERVATION). *Suppose M is a closed term and Γ is a consistent, top-level type environment in which all top-level identifiers have justified typings $\vdash \mathcal{M} : \Gamma$.*

- *If $\Gamma \vdash M : A$ and $M \triangleright N$ then $\Gamma \vdash N : A$.*
- *If $\Gamma, M : A \vdash$ and $M \triangleright N$ then $\Gamma, N : A \vdash$.*

THEOREM 6.12 (SYNTACTIC SOUNDNESS). *Suppose M is a closed term and Γ is a consistent, top-level type environment in which all top-level identifiers have justified typings $\vdash M : \Gamma$.*

- *If M is well-typed in Γ , then M does not go wrong.*
- *If M is ill-typed in Γ , then M does not evaluate.*

7 CONCLUSION AND RELATED WORK

We have introduced *two-sided type systems* which are sequent calculi for typing formulas, made possible, for λ -calculi through a new function type $A \multimap B$. We have shown several ways in which these calculi can be considered sound, and illustrated how left-sided rules can be added to a constrained type system. We also investigated the internalisation of negation, and a one-sided system without a true negatives theorem. Many of the basic constants of the programming languages we use today have necessary requirements on their inputs. However, traditional type systems largely ignore this basic aspect of their behaviour. Two-sided systems can use this additional dimension, and enable simple reasoning about program incorrectness as well as program correctness.

Typing Contexts Beyond Variables. Many type systems for higher-order program verification generalise the typing context to allow logical formulae for the purpose of recording path conditions. For example, Liquid Types [Rondon et al. 2008; Vazou et al. 2015, 2013] and the systems of Terauchi [2010] and Unno and Kobayashi [2009] place the Boolean valued expression M (or an equivalent formula) into the context when proving that the ‘then’-branch of a conditional, if M then N else P , is correctly typed. In a two-sided system, this can be managed very naturally by introducing separate types for true and false (as we have in our system of Section 6) and assuming $M : \text{true}$. A rather different use of the typing context was made by Curien and Herbelin [2000]. Their system is a kind of sequent calculus in which the formulas on the left of the turnstile comprise variable typings and a stack of terms (cf. Krivine machines), which is thought of as an evaluation context.

Constrained Type Systems. Constrained type systems and inclusion constraints go back at least to the work of Mitchell [1984]. A unified treatment is given by Odersky et al. [1999]. Our system takes inspiration from the one-sided system of Aiken et al. [1994] (see also Marlow and Wadler [1997]), though their constraints are far more expressive. Our approach to consistency of subtype constraints is an adaptation of the purely syntactic approach of Eifrig et al. [1995]. It is not clear how well our algorithm would scale in practice, ideally one would like some guarantees on the size of types or the efficiency of inference, such as features in more recent works by Dolan and Mycroft [2017] and Jones and Ramsay [2021]. None of these systems has a true positives theorem.

Success Typing. Our original motivation was to try to better understand the very effective *success typing* paradigm for Erlang [Lindahl and Sagonas 2006]. Work by Jakob and Thiemann [2015] provides a useful view through their falsification type system, in which one constructs a logical formula used to describe inputs that guarantee to crash the function. The formalisation of López-Fraguas et al. [2018] is also enlightening and they moreover extend the system with polymorphism.

Success Types are perhaps the best known example of a type-based true positives theorem, ‘ill-typed programs always fail’ [Sagonas 2010]. They are so named because they over-approximate the successes of expressions: if A is a success type and $M : A$ then M may succeed in evaluating to an A , but may also go wrong or diverge. Hence why we named our alternative semantics after this paradigm. Note, the ‘ill-typed’ of the slogan really means untypable in our sense.

A key feature is a bespoke function type. It is defined in [Lindahl and Sagonas 2006] as follows:

A success typing of a function f is a type signature $(\vec{\alpha}) \rightarrow \beta$, such that whenever an application $f(\vec{p})$ reduces to a value v , then $v \in \beta$ and $\vec{p} \in \vec{\alpha}$.

Thus, the *success arrow*, let us write $\rightarrow_s$, can be understood as having the right rule shown to the side. That is, it contains a component of necessity, but restricted to Ok on the right. Hence, with only this

$$\frac{\Gamma, M : \text{Ok} \vdash x : \text{Ok} \quad \Gamma, M : \text{Ok} \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow_s B, \Delta}$$

arrow, one can refute that an application MN has a type A only when $A = \text{Ok}$. Yet, despite this, success types appear to be highly effective in practice. One reason could be the following: if an application

MN fails to evaluate to an A , then there are three possibilities, it could go wrong, it could diverge, or it could evaluate to something else. In the first two cases, one can attempt to refute that it has type Ok, and in the third, one can attempt to affirm that it has a type that is disjoint from A instead. Thus, it is likely that one can get quite far with only this form of necessity.

Logics for Incorrectness. Incorrectness logic of O'Hearn [2019] has sparked a new interest in systems for reasoning about program behaviour that enjoy true positives theorems. Subsequent work has extended its scope [Raad et al. 2022], and demonstrated its real-world effectiveness [Le et al. 2022]. A key feature of these logics is *under-approximation* as a means to achieve true positives, and this allows for a more specific notion of 'positive', which excludes divergence. In incorrectness logic, states in a postcondition must be *reachable*. This leads to the need to reason about termination, but it also allows for an analysis to dynamically drop disjunctions in order to scale well. A interesting alternative is the Outcome Logic of Zilberstein et al. [2023]. Here there is a distinction between Boolean disjunction and outcome disjunction. The former has true as annihilator, but the latter does not, and this allows for the dropping of outcomes for efficiency.

Necessity. The foundation of our system is not under-approximation, but necessity. When $M : A$ is provable, type A is an *over-approximation* of M . This makes the system closer to the works of, on the one hand, Cousot et al. [2013, 2011], and on the other, Mackay et al. [2022]. The former considers the problem of inferring *necessary preconditions* in order to detect contract violations that will lead to assertion failures. Specifications are given as code assertions and various analyses based on abstract interpretation are showcased for efficient inference. The latter uses necessity as a means to specify the *robustness* of module specifications in object-oriented programming. Three novel necessity operators are added to an assertion language equipped with certain object capabilities, and a logic is developed for proving that a module satisfies its necessity specifications.

Type Systems with Complement. A complement operator appears in the system of Aiken et al. [1994] and a negation in the work of Parreaux and Chau [2022], though neither has a true positives theorem. In the highly expressive dependent type system of Unno et al. [2017], none of the programs can go wrong, but the system is nevertheless very well-equipped for incorrectness reasoning, and includes sophisticated means for reasoning about recursion and tracking divergence. Since their system can express the complements of arbitrary types, it should be possible to encode a version of our necessity arrow. One can think of our one-sided system of Section 5 as a kind of sweet spot for partial correctness, in which the theory and automation become particularly straightforward.

DATA AVAILABILITY STATEMENT

A long version of the paper is available, which contains all definitions and proofs that were omitted here due to space constraints [Ramsay and Walpole 2023].

ACKNOWLEDGMENTS

The first author is very grateful for an award from Meta Research. This paper was much improved due to the advice of the POPL'24 anonymous reviewers, and we especially thank our shepherd

Sophia Drossopoulou. The work has also benefited from many useful discussions with colleagues in the Programming Languages Research Group at the University of Bristol.

REFERENCES

- Alexander Aiken, Edward L. Wimmers, and T. K. Lakshman. 1994. Soft Typing with Conditional Types. In *Conference Record of POPL '94: 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Portland, Oregon, USA, January 17-21, 1994*. 163–173. <https://doi.org/10.1145/174675.177847>
- Julian Bradfield and Igor Walukiewicz. 2018. *Handbook of Model Checking*. Springer International Publishing, Chapter The mu-calculus and model checking, 871–919. https://doi.org/10.1007/978-3-319-10575-8_26
- Patrick Cousot, Radhia Cousot, Manuel Fähndrich, and Francesco Logozzo. 2013. Automatic Inference of Necessary Preconditions. In *Verification, Model Checking, and Abstract Interpretation*, Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 128–148. https://doi.org/10.1007/978-3-642-35873-9_10
- Patrick Cousot, Radhia Cousot, and Francesco Logozzo. 2011. Precondition Inference from Intermittent Assertions and Application to Contracts on Collections. In *Verification, Model Checking, and Abstract Interpretation*, Ranjit Jhala and David Schmidt (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 150–168. https://doi.org/10.1007/978-3-642-18275-4_12
- Pierre-Louis Curien and Hugo Herbelin. 2000. The duality of computation. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, Martin Odersky and Philip Wadler (Eds.). ACM, 233–243. <https://doi.org/10.1145/351240.351262>
- Stephen Dolan and Alan Mycroft. 2017. Polymorphism, Subtyping, and Type Inference in MLsub. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (Paris, France) (POPL '17)*. Association for Computing Machinery, New York, NY, USA, 60–72. <https://doi.org/10.1145/3009837.3009882>
- H.-D. Ebbinghaus, J. Flum, and W. Thomas. 1994. *Mathematical Logic*. Springer. <https://doi.org/10.1007/978-1-4757-2355-7>
- Jonathan Eifrig, Scott Smith, and Valery Trifonov. 1995. Sound Polymorphic Type Inference for Objects. In *Proceedings of the Tenth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications (Austin, Texas, USA) (OOPSLA '95)*. Association for Computing Machinery, New York, NY, USA, 169–184. <https://doi.org/10.1145/217838.217858>
- Robert Jakob and Peter Thiemann. 2015. A Falsification View of Success Typing. In *NASA Formal Methods*, Klaus Havelund, Gerard Holzmann, and Rajeev Joshi (Eds.). Springer International Publishing, Cham, 234–247. https://doi.org/10.1007/978-3-319-17524-9_17
- Eddie Jones and Steven Ramsay. 2021. Intensional Datatype Refinement: With Application to Scalable Verification of Pattern-Match Safety. *Proc. ACM Program. Lang.* 5, POPL, Article 55 (jan 2021), 29 pages. <https://doi.org/10.1145/3434336>
- Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Finding Real Bugs in Big Programs with Incorrectness Logic. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 81 (apr 2022), 27 pages. <https://doi.org/10.1145/3527325>
- Tobias Lindahl and Konstantinos Sagonas. 2006. Practical Type Inference Based on Success Typings. In *Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (Venice, Italy) (PPDP '06)*. Association for Computing Machinery, New York, NY, USA, 167–178. <https://doi.org/10.1145/1140335.1140356>
- Francisco J. López-Fraguas, Manuel Montenegro, and Gorka Suárez-García. 2018. Polymorphic success types for Erlang. In *LPAR-22. 22nd International Conference on Logic for Programming, Artificial Intelligence and Reasoning (EPiC Series in Computing, Vol. 57)*, Gilles Barthe, Geoff Sutcliffe, and Margus Veanes (Eds.). EasyChair, 515–533. <https://doi.org/10.29007/w2m2>
- Julian Mackay, Susan Eisenbach, James Noble, and Sophia Drossopoulou. 2022. Necessity Specifications for Robustness. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 154 (oct 2022), 30 pages. <https://doi.org/10.1145/3563317>
- Simon Marlow and Philip Wadler. 1997. A Practical Subtyping System for Erlang. In *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming (Amsterdam, The Netherlands) (ICFP '97)*. Association for Computing Machinery, New York, NY, USA, 136–149. <https://doi.org/10.1145/258948.258962>
- John C. Mitchell. 1984. Coercion and Type Inference. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (Salt Lake City, Utah, USA) (POPL '84)*. Association for Computing Machinery, New York, NY, USA, 175–185. <https://doi.org/10.1145/800017.800529>
- Martin Odersky, Martin Sulzmann, and Martin Wehr. 1999. Type Inference with Constrained Types. *TAPOS* 5, 1 (1999), 35–55. [https://doi.org/10.1002/\(SICI\)1096-9942\(199901/03\)5:1<35::AID-TAPO4>3.0.CO;2-4](https://doi.org/10.1002/(SICI)1096-9942(199901/03)5:1<35::AID-TAPO4>3.0.CO;2-4)
- Peter W. O'Hearn. 2019. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (dec 2019), 32 pages. <https://doi.org/10.1145/3371078>
- Lionel Parreaux and Chun Yin Chau. 2022. MLstruct: Principal Type Inference in a Boolean Algebra of Structural Types. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 141 (oct 2022), 30 pages. <https://doi.org/10.1145/3563304>
- Azalea Raad, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Concurrent Incorrectness Separation Logic. *Proc. ACM Program. Lang.* 6, POPL, Article 34 (jan 2022), 29 pages. <https://doi.org/10.1145/3498695>

- Steven Ramsay and Charlie Walpole. 2023. III-Typed Programs Don't Evaluate. <https://doi.org/10.48550/arXiv.2307.06928> [cs.PL]
- Patrick Maxim Rondon, Ming Kawaguchi, and Ranjit Jhala. 2008. Liquid types. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*. 159–169. <https://doi.org/10.1145/1375581.1375602>
- Konstantinos Sagonas. 2010. Using Static Analysis to Detect Type Errors and Concurrency Defects in Erlang Programs. In *Functional and Logic Programming*, Matthias Blume, Naoki Kobayashi, and Germán Vidal (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 13–18. https://doi.org/10.1007/978-3-642-12251-4_2
- Tachio Terauchi. 2010. Dependent types from counterexamples. In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*. 119–130. <https://doi.org/10.1145/1706299.1706315>
- Hiroshi Unno and Naoki Kobayashi. 2009. Dependent type inference with interpolants. In *Proceedings of the 11th International ACM SIGPLAN Conference on Principles and Practice of Declarative Programming, September 7-9, 2009, Coimbra, Portugal*. 277–288. <https://doi.org/10.1145/1599410.1599445>
- Hiroshi Unno, Yuki Satake, and Tachio Terauchi. 2017. Relatively Complete Refinement Type System for Verification of Higher-Order Non-Deterministic Programs. *Proc. ACM Program. Lang.* 2, POPL, Article 12 (dec 2017), 29 pages. <https://doi.org/10.1145/3158100>
- Niki Vazou, Alexander Bakst, and Ranjit Jhala. 2015. Bounded refinement types. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming, ICFP 2015, Vancouver, BC, Canada, September 1-3, 2015*. 48–61. <https://doi.org/10.1145/2784731.2784745>
- Niki Vazou, Patrick Maxim Rondon, and Ranjit Jhala. 2013. Abstract Refinement Types. In *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*. 209–228. https://doi.org/10.1007/978-3-642-37036-6_13
- Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (Gothenburg, Sweden) (ICFP '14)*. 269–282.
- Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press. <https://doi.org/10.7551/mitpress/3054.001.0001>
- A.K. Wright and M. Felleisen. 1994. A Syntactic Approach to Type Soundness. *Information and Computation* 115, 1 (1994), 38–94. <https://doi.org/10.1006/inco.1994.1093>
- Noam Zilberstein, Derek Dreyer, and Alexandra Silva. 2023. Outcome Logic: A Unifying Foundation for Correctness and Incorrectness Reasoning. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 93 (apr 2023), 29 pages. <https://doi.org/10.1145/3586045>

Received 2023-07-11; accepted 2023-11-07